professor.py, line by line

How the Alpha Rodeo Hidden Hand bot works: a study guide to read, annotate and explain

Team	Alpha Rodeo
Member	Jaiv Patel (26075057)
Mentor	Arpit Gupta
Final upload	heist/test_tt01_plan8w3.py as uploaded = matias (v15), entry #862, 1,078 lines
Event	Hidden Hand, The Quant Club, IIT BHU
Prepared	Thursday 8 October 2026

Code colours: **keywords**, CONSTANTS, **function names**, numbers, "strings", # *comments*, *docstrings*. Grey numbers on the left are the line numbers of heist/professor.py.

Contents

Chapters, then every code block of the walkthrough by its line numbers.

How to use this guide . . . 5

The bot on one page . . . 6

The ten lines to remember . . . 6
Key numbers . . . 6
Live record of this file (entry #862) . . . 6

1. The game in one page . . . 8

Cards and the stock . . . 8
A deal has 5 rounds . . . 8
Settlement and score . . . 8
What the bot sees: the Obs object . . . 8
The three methods the engine calls . . . 9
A tiny example . . . 9

2. The big picture . . . 10

The problem in one line . . . 10
Job 1: inference (learning) . . . 10
Job 2: decisions (planning) . . . 10
Moves . . . 10
Two safety layers . . . 10
Where each part lives . . . 11
Every function at a glance . . . 11

3. Five ideas, with worked examples . . . 14

3.1 The value of the stock given O (_ev, line 632) . . . 14
3.2 How the opponent values the stock (coefs, line 66) . . . 14
3.3 The 216 hypotheses (_init, line 394) . . . 14
3.4 Learning from an opponent quote: the Kalman filter (_see, lines 531–553) . . . 14
3.5 Learning from an answer to my quote (_see, lines 554–590) . . . 15
3.6 Weights and pruning (lines 553, 583, 427) . . . 15
3.7 Choosing a price (_quote, line 873) . . . 15
3.8 The utility (util, line 61) . . . 15
3.9 The swap (_move, line 1037) . . . 15

4. Header and constants (lines 1–50) . . . 17

1–9 The required header, our version note and the imports . . . 17
11–14 Basic numbers . . . 17
16–29 Opponent styles and their prior probabilities . . . 17
31–48 Noise, speed and decision settings . . . 18

49–50 Hypothesis fields . . . 19

5. Maths helpers (lines 53–170) . . . 20

53–58 The normal curve . . . 20
61–63 The utility . . . 20
66–82 The opponent's value as a straight line . . . 21
85–86 implied . . . 22
89–105 pick_q: which of my quotes a reader uses . . . 22
108–120 ekf: one Kalman step (the readable version) . . . 22
123–138 mix_into: hedging against outliers . . . 23
141–170 ekf_mix: the same, fused (the one actually used) . . . 24

6. The house-reader attack (lines 173–381) . . . 26

What this part is for . . . 26
173–187 Constants of the attack . . . 26
190–196 _lots: the sharp's trade size . . . 26
199–210 _Reader: its memory . . . 27
212–224 invert: their hand sum from one quote . . . 27
226–249 observe: is this quote consistent with the earlier ones? . . . 28
251–271 sync: feed new quotes and verify predictions . . . 29
273–276 active . . . 29
278–282 value: V when S is known . . . 30
284–293 their_base: their value for a given S and my last quote P . . . 30
295–306 events_left: who quotes next, until the end of the deal . . . 30
308–363 plan: exact look-ahead against the house bot . . . 31
365–381 quote and respond against an identified reader . . . 32

7. The Bot: inference (lines 384–716) . . . 34

384–391 Creating the bot . . . 34
393–425 _init: the prior and the 216 hypotheses . . . 34
427–431 _prune . . . 35
433–453 _state: the public numbers as the opponent saw them in round r . . . 35
455–467 _before: my position and Q before slot j . . . 36
469–477 _ks: the sign swap-expecting bots use . . . 37
479–495 _lastq: my last quote, carried forward . . . 37
497–514 _see, part 1: one observation . . . 38
515–530 _see, part 2: per-hypothesis set-up . . . 38
531–553 _see, part 3: learning from their quote . . . 39
554–590 _see, part 4: learning from their answer to my quote . . . 39
592–599 _see_swap: the opponent swapped . . . 40
601–630 _sync: process everything new, in order . . . 41
632–643 _ev: E[V | O] = α + β·O . . . 41
645–655 _opp_swap (not used any more) . . . 42
657–664 _mo: our best guess of O . . . 42

666–678 _p_reader . . . 43
680–716 _exact_reader: does the opponent fit the house reader formula? . . . 43

8. The Bot: planning and decisions (lines 718–1078) . . . 45

718–728 _slots: who quotes in each remaining slot . . . 45
730–746 _bel: their value under one hypothesis . . . 45
748–760 _roll: set-up . . . 46
761–789 _roll: my quotes . . . 46
790–804 _roll: their quotes . . . 47
806–818 _plan_setup . . . 47
820–838 _public (not used) and _plain_fair (the fallback) . . . 48
840–871 The three methods the engine calls . . . 48
873–890 _quote, part 1: set-up . . . 49
892–919 _quote, part 2: their answer model . . . 50
921–927 _quote, part 3: the immediate expected profit . . . 50
929–942 _quote, part 4: stage 1, the best price for this trade alone . . . 51
943–960 _quote, part 5: stage 2 candidates . . . 51
961–985 _quote, part 6: score each candidate with the rest of the deal, then nudge . . . 52
987–1003 _safe_quote . . . 52
1005–1035 _respond: buy, pass or sell? . . . 53
1037–1078 _move: never peek, swap only in round 5 . . . 54

9. Every parameter in one table . . . 56

10. How we got here . . . 58

Version history . . . 58
Ideas tested live and dropped . . . 58
How we tested . . . 58

11. Questions the organisers may ask . . . 60

12. Glossary . . . 63

Notes . . . 64

How to use this guide

This guide explains **heist/professor.py**, our final bot, line by line: it is **matias (v15)**, uploaded as heist/test_tt01_plan8w3.py and live as entry #862 (heist/matias.py is the same code; only line 4 differs). matias is marseille (v12) with four small changes: line 37 (swap in round 5 whenever the expected gain is positive; marseille needed 4 points; manila, v13), line 1069 (decide the swap on a mix of the position we hold now and the position we expect to end with; pamplona, v14, made it 50/50 and matias 70/30 towards the position we hold now) and line 39 (plan with the 8 likeliest hypotheses instead of 6; matias). Everything else is marseille's code. Line numbers in this guide are those of the uploaded file (1,078 lines).

Read it in this order:

- **The one-page summary and chapters 1–2** tell the whole story in a few pages. Read them twice.
- **Chapter 3** explains the five ideas you need (the value formula, the hypotheses, the Kalman filter, learning from answers, the planner). Its small examples were computed by running the real functions.
- **Chapters 4–8** are the line-by-line walkthrough. Each grey box is a piece of the file with its line numbers. Under it, every line or group of lines is explained ("L54" means line 54).
- **Chapters 9–12** list every parameter, the version history, questions the organisers may ask (with answers) and a glossary.

Two kinds of coloured boxes:

- **SAY IT LIKE THIS**: one or two sentences you can say out loud.
- **WHY**: why we chose it, and the test that decided it.

Every chapter ends with a box for your notes, and there are blank note pages at the end.

The bot on one page

The ten lines to remember

1. V = my hand + the board + **O**, the opponent's hand sum. O is the only real unknown (about 35 ± 8 at the start).
2. We keep **216 hypotheses** about how the opponent forms its price: naive, four kinds of reader, or blend quoter, times how honest its quotes are (lam), how it reacts to my trades (tau), whether it expects me to swap (kappa), and how much edge it needs to trade (2 or 7 points).
3. Each hypothesis has a Gaussian belief over (O, t), where t is how much the opponent trusts my quotes, plus a log-weight.
4. Their quote → **Kalman update** (under each hypothesis the quote is a straight line in O and t). Their answer to my quote → **truncated-normal update** (it tells us an interval). Their swap → the belief shifts and widens.
5. Weights add the log-probability each hypothesis gave to what happened (Bayes' rule). 10% of quotes and 6% of answers may be outliers. Hypotheses 6 log-units behind the best are dropped.
6. Value: **E[V | O] = α + β·O**.
7. Quote: stage 1 finds the best immediate profit (including **adverse selection**); stage 2 adds a play-out of the rest of the deal under the 8 likeliest hypotheses, through a square-root-like **utility** (like the score). Then the final quote is **70% planner + 30% safe quote** (their last quote, 1.3 towards our value).
8. Answer: buy, pass or sell, compared with the same yardstick.
9. Move: **never peek**. Swap at most once, **only in round 5**: our lowest card if we expect to end long, our highest if short, whenever the expected gain is positive, judged on 0.7 × our current position + 0.3 × our expected final position.
10. An exact attack on the house bots (off against real teams), try/except fallbacks everywhere, nothing kept between deals, standard library only (heapq, math).

Key numbers

Thing	Value	Where
hypotheses	216	_init, line 394
hypotheses used to plan	8 (NPLAN; 6 before matias)	line 39
pruning distance	6 log-units = 403×	line 36
utility curvature	RISK = 60	line 44
nudge	30% (SAFE_W), step 1.3 (SAFE_D)	lines 45–46
prior: does not read my quotes	30% (P_NAIVE)	line 17
opponent take threshold	2 points (75%) or 7 (25%)	line 26
swap bar in round 5	0: any positive expected gain (marseille: 4)	line 37
swap decided on	0.7 × current + 0.3 × expected final position	line 1069
speed (local)	median 0.20 ms, 99% under 1 ms per call	ledger

Live record of this file (entry #862)

Against the uploaded pamplona (#713) on the same 29 opponents: better against 22, worse against 7, median +0.18 (mean +0.16 ± 0.18; +0.23 ± 0.11 without one outlier). Against the leaders on the same opponents: Imposters

+0.25, PaperHands +0.37, TeenPatti +1.25, Genz7 +2.39, MNC +2.78; head to head it beat Imposters' newest bot by +1.94. Before it, pamplona beat manila by +0.40 ± 0.13 and manila beat marseille by +0.10 ± 0.05.

Your notes

1. The game in one page

Cards and the stock

- One 52-card deck. A card is worth its rank: A = 1, 2 to 10, J = 11, Q = 12, K = 13. There are four copies of every rank, so the whole deck adds up to 4 × 91 = **364** (you will see 364 in the code).
- Each player gets 5 private cards, the **hand**. Five more cards are the face-down **board**. The other 37 cards are the **stack**, used only for swaps.
- The stock settles at **V = my hand + the opponent's hand + the whole board**: 15 cards, 105 on average.

A deal has 5 rounds

- At the start of round r, board card r − 1 is turned face up: round 1 shows none, round 5 shows four. The fifth board card is never shown before settlement.
- Each round starts with a **move**, chosen by both bots at the same time: nothing, **PEEK** (pay 3, see one random card of the opponent's hand) or **SWAP** (free, once per deal: throw one card face up, draw a fresh one from the stack).
- Then two **quotes**. The **maker** names one price p. The **taker** answers: buy up to 2 lots at p + 2, sell up to 2 lots at p − 2, or pass. Then the roles swap.
- Nobody may hold more than **10 lots** long or short; the engine cuts any trade that would break this. A maker that fails to quote is quoted at 105.

Round	Board shown	Move phase	First maker	Second maker
1	0 cards	both choose	seat 0	seat 1
2	1 card	both choose	seat 1	seat 0
3	2 cards	both choose	seat 0	seat 1
4	3 cards	both choose	seat 1	seat 0
5	4 cards	our swap happens here	seat 0	seat 1

So there are 10 quotes per deal, the "slots" 0 to 9 (slot s is in round s // 2 + 1). Each bot quotes 5 times and answers 5 times.

Settlement and score

- PnL = cash from trades + position × V − money spent on peeks.
- Every deal is played twice with the seats swapped (same cards), so the luck of the cards cancels. For each pair, p = the PnL over both plays; the score is the average of **sign(p) × √|p|**. Winning by 100 counts 10, winning by 25 counts 5, so many steady wins beat one huge win.
- A call that takes more than 50 ms is replaced by a fallback answer. Our target is far below that.

What the bot sees: the Obs object

Field	Meaning
round	1 to 5
hand	my 5 cards now (ranks 1 to 13)
board	the board cards face up so far
opp_known	opponent cards I peeked that they still hold (always empty for us: we never peek)
unseen	unseen[r − 1] = copies of rank r I have not seen anywhere (not in my hand, the board, the discards or my peeks)
position	lots I hold: + long, − short

Field	Meaning
cash, spent	money from my trades; money paid for peeks
tape	every quote so far, oldest first: (round, maker "me" or "opp", price, qty). qty is what the TAKER did: +n = bought n at price + 2, −n = sold n at price − 2, 0 = passed
discards	every swapped-out card: (round, who, rank)
peeks	every peek: (round, who); nobody learns which card was seen

The three methods the engine calls

- `move(obs)` returns None, "PEEK" or ("SWAP", i).
- `quote(obs)` returns a price (the engine clamps it to 0..195 and rounds to 0.01).
- `respond(obs, price)` returns −2 to +2 lots.

A tiny example

I quote 100. The opponent buys 2 lots: it pays 102 each and I am now short 2. The final V is 110. My PnL = 2 × (102 − 110) = −16. The opponent bought because it knew its hand was high. Losing to the better-informed side like this is called **adverse selection**, and much of the code is about avoiding it.

Your notes

2. The big picture

The problem in one line

V = my hand (known) + the board (partly known) + **O**, the opponent's hand sum, which is the only real unknown. So the game is two things: (1) estimate O, and (2) know what the opponent believes, because that decides how it trades against my prices.

Job 1: inference (learning)

We keep a list of 216 **hypotheses**, each a possible opponent style: does it read my quotes, how honest are its own quotes, how much edge does it need before it trades, and so on. Each hypothesis carries its own Gaussian belief about (O, t) (t = how much the opponent trusts my quotes) and a weight that says how well it has predicted the opponent so far. Every opponent quote and every answer to my quotes updates all of them.

Job 2: decisions (planning)

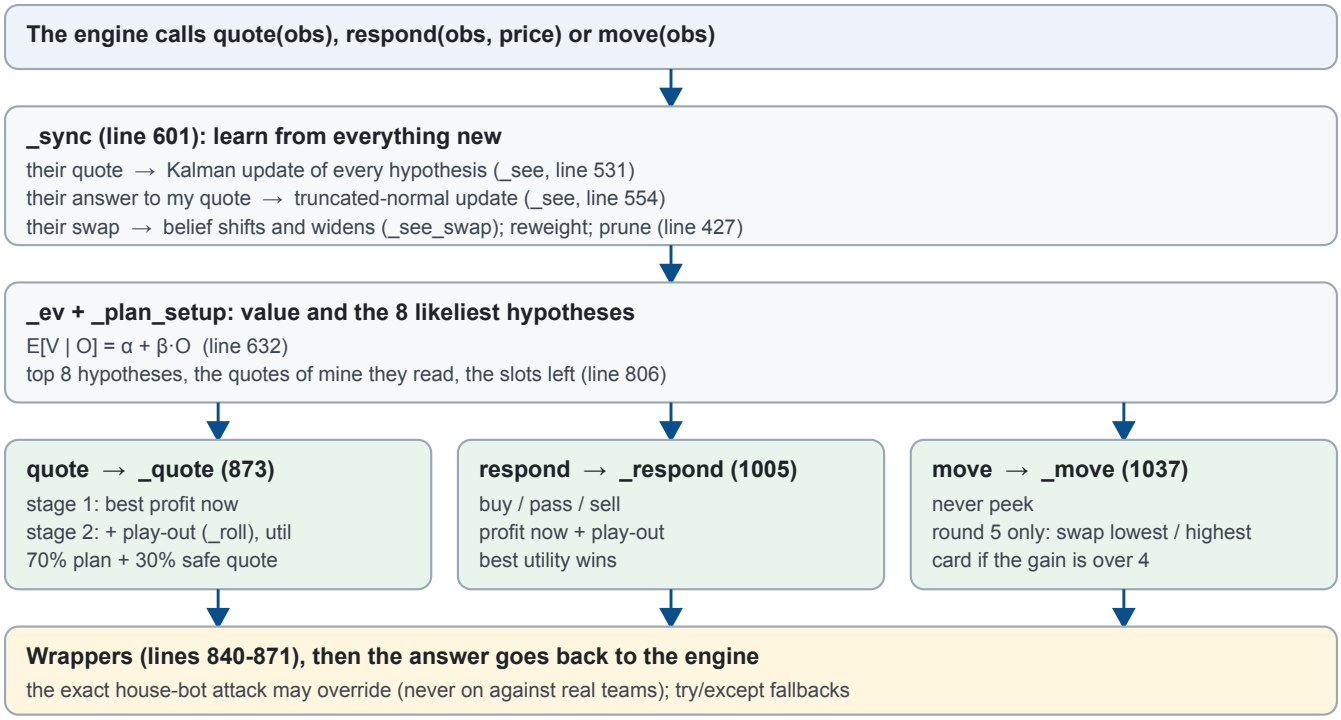
For each possible action (a price to quote; buy, pass or sell) we compute the expected profit now plus a quick simulation of the rest of the deal under the 8 most likely hypotheses, score the total with a concave utility that mirrors the √ in the score, and take the best. Our final quote is then moved 30% of the way towards a **safe quote**: the opponent's last quote, moved 1.3 towards our value.

Moves

We never peek. We swap at most once, only in round 5: our lowest card if we expect to end long, our highest if we expect to end short, whenever the expected gain is positive. The decision uses 0.7 × the position we hold now + 0.3 × the position we expect to end with.

Two safety layers

- An exact attack on the house bots "reader" and "sharp" (class _Reader). It switches on only when the opponent's quotes match the house formula to the cent, and off at the first surprise. Against real teams it stays off.
- Every engine call is wrapped in try/except with a simple fallback, so a bug can never crash a deal.



Where each part lives

Lines	Part
1–50	header, version note, imports, constants
53–170	maths helpers: normal curve, utility, the opponent's value formula, Kalman update
173–381	_Reader: exact attack on the house reader and sharp bots
384–431	Bot: creation, the 216 hypotheses, pruning
433–716	Bot: inference (public state, learning from quotes, answers and swaps)
718–818	Bot: planning helpers (slots, beliefs, play-outs)
820–871	the three engine methods with their fallbacks
873–985	_quote: choosing a price
987–1003	_safe_quote: the nudge target
1005–1035	_respond: buy, pass or sell
1037–1078	_move: the round-5 swap

Every function at a glance

Every function in the file, in file order: what it is for and what it gives back. The walkthrough has a full card for each.

Function	Line	What it does	Returns
`Phi`	53	The standard normal cumulative probability: the chance that a standard normal value is at most x	A number between 0 and 1
`pdf`	57	The standard normal density at x	A number of 0 or more
`util`	61	Concave, score-shaped utility of a deal's profit	A number with the sign of x
`coefs`	66	The opponent's fair value as a straight line in O and t	A tuple (A0, A1, B0, B1)
`implied`	85	The hand sum of ours that quote q implies to an opponent with an average hand	A number (a hand sum)
`pick_q`	89	Which of our quotes a reader reads now	A pair (quote or None, cap or None)
`ekf`	108	One Kalman step (readable reference)	A tuple (mean O, mean t, P00, P01, P11); h itself is not changed
`mix_into`	123	Blend new and old belief (reference)	Nothing; changes h in place
`ekf_mix`	141	The Kalman step + outlier blend actually used	Nothing; updates h's belief in place
`_lots`	190	The house sharp bot's trade size for an edge e	Signed lots: 0, ±1 or ±2
`_Reader · __init__`	199	Exact attack on house bots: its memory	Nothing
`invert`	213	Hand sum from one house-bot quote	S (a whole number from 5 to 65), or None if nothing fits within 0.0051
`observe`	226	Is this quote consistent with the run?	Nothing; updates S, nq and nread
`sync`	251	Feed new quotes; verify predictions	Nothing
`active`	273	Decides whether the exact attack may act now	True or False
`value`	278	The stock's expected value once the opponent's hand sum S is known	A number
`their_base`	285	The house bot's fair value for hand sum S, given our last quote P (or None)	A number

Function	Line	What it does	Returns
`events_left`	296	Lists the remaining quotes of the deal, starting with ours now	A list of (who, round), who being "me" or "opp"
`plan`	308	Exact look-ahead against a house bot	A pair (best total profit, the best quote to make now)
`_Reader.quote`	365	The attack's quote and answer	quote: the price to quote. respond: +2, −2 or 0
`Bot.__init__`	385	A fresh bot for each deal	Nothing
`_init`	394	Build the 216 hypotheses	Nothing; sets self.H and self.obar
`_prune`	427	Drops hypotheses more than 6 log-units behind the best and shifts the weights so the best is 0	Nothing
`_state`	434	The public numbers as the opponent saw them in round r	A tuple: board sum, hidden board cards, their peeks, our peeked sum, cards unknown to them, total of those cards, average unknown card
`_before`	456	Position and Q before a given slot	A pair (our position, Q = net lots we bought at their quotes)
`_ks`	470	The direction a swap-expecting opponent assumes we will move V: the sign of our position while we can still swap	+1, −1 or 0
`_lastq`	480	Our last quote before slot j, carried forward for new board cards and our swaps the way a blend quoter carries it	A number, or None if we have not quoted
`_see`	497	Learn from one quote or answer	Nothing; updates self.H
`_see_swap`	592	Updates every belief after the opponent swaps: O moves by (average new card − thrown card) and becomes more uncertain	Nothing
`_sync`	601	Catch up on everything new, in order	Nothing
`_ev`	632	E[V given O] = alpha + beta·O	A pair (alpha, beta) with E[V / O] = alpha + beta·O
`_opp_swap`	646	(unused) price in the opponent's swap	A number
`_mo`	657	Our single best guess of O: the weighted average over all hypotheses	A number
`_p_reader`	666	Chance the opponent is a confident reader	A pair (p_reader, p_raw)
`_exact_reader`	680	Does the opponent fit the house formula?	t, or None
`_slots`	720	Lists the remaining quote slots and who is the maker in each	A list of (we are the maker?, round)
`_bel`	730	The opponent's fair value under one hypothesis with point values, reading our quote q	A pair (fair value, slope in O)
`_roll`	748	Quick play-out of the rest of the deal	A pair (our profit relative to V, our final position)
`_plan_setup`	806	Shared setup for every decision	A tuple (st, hs, ws, qe, slots, swapped)
`_public`	822	Card-counting fallback value	A number each
`quote`	840	The engine's entry points, with fallbacks	quote: our price. respond: lots to buy (+) or sell (−), at most 2. move: None or ("SWAP", card index)
`_quote`	873	Choose our price as the maker	The price, between 0 and 195

Function	Line	What it does	Returns
`resp_dist`	894	Model of the opponent's answer (cached)	A tuple (mean f, sd, cov(O, fair) / sd)
`imm`	921	Expected profit of a price, with adverse selection	A pair (expected profit, their fair value)
`_safe_quote`	988	The safe quote the nudge aims at	A price, or None if the opponent has not quoted yet
`_respond`	1005	Choose buy, pass or sell	Lots: 0, +nbuy or −nsell (each at most 2, limited by the position cap)
`_move`	1037	The round-5 swap; never peek	None, or ("SWAP", index of the card to throw)

SAY IT LIKE THIS

"Our bot is a Bayesian trader. It keeps 216 guesses about the opponent's style, each with a belief about the opponent's hand that a Kalman filter updates from every quote and answer. To act, it scores each option by profit now plus a play-out of the rest of the deal under the eight likeliest guesses, through a utility shaped like the score, and then hides a little by moving its quote 30% towards the opponent's last quote."

Your notes

3. Five ideas, with worked examples

All numbers in this chapter come from running the functions in pamplona.py.

3.1 The value of the stock given O (_ev, line 632)

The unknowns for me are the opponent's 5 cards (sum O) and the hidden board cards. If I knew O, the hidden board cards would be random cards from the rest of my unseen pool. So

E[V | O] = α + β × O, with α = my hand + visible board + (hidden board cards) × (unseen sum) / (unseen count − 5) and β = 1 − (hidden board cards) / (unseen count − 5).

Example, round 1, my hand sum 40: there are 47 unseen cards worth 364 − 40 = 324. α = 40 + 5 × 324 / 42 = 78.57 and β = 0.881. If O = 35 then V ≈ 109.4; if O = 45 then V ≈ 118.2.

Why β is below 1: if the opponent's hand is high, the cards left for the board are a little lower on average.

3.2 How the opponent values the stock (coefs, line 66)

From the opponent's side, my hand is the unknown. A **naive** opponent uses only its own hand and the public cards. In round 1 its fair value is 77.45 + 0.787 × O (105.0 if O = 35, 112.9 if O = 45).

A **reader** also reads my last quote q. It works out the hand my quote implies (function implied) and moves its fair value by t × (implied hand − average hand), where t is its trust in me. Example: my quote 115 implies a hand of 45, ten above average; a reader with t = 0.6 and O = 35 moves from 105 to 111.

So under every hypothesis, the opponent's fair value is a straight line in O and t:

fair = A0 + A1·t + (B0 + B1·t)·O (plus two small extra terms, tau·Q and kappa·ks).

That straight-line form is what makes the Kalman filter work.

3.3 The 216 hypotheses (_init, line 394)

Family	Rule	Prior share	Count
naive	does not read my quotes	0.30 × 0.88	24
reader RAW	reads my last quote as it is	0.70 × 0.55 × 0.88	24
reader CLAMP	caps the hand my quote implies to 5..65	0.70 × 0.20 × 0.88	24
reader SANE	ignores quotes implying a hand outside 12..58	0.70 × 0.08 × 0.88	24
reader CUR	also reads the price it is answering	0.70 × 0.17 × 0.88	24
blend (BLD)	quotes a mix of its own value and my last quote	0.12	96

Inside each family the hypotheses differ in:

- **lam**: how much of its own hand its quotes show (1, 0.7, 0.4 or 0; 0 means its quotes say nothing).
- **tau**: how much its value moves per lot I buy from it (0, 1.5 or 3; only for fully honest quoters).
- **kappa**: 0 or 3.9, its expectation that I will swap away a bad card.
- **threshold**: it needs 2 or 7 points of edge before it trades.
- blend quoters also have a **skew**: 0 or 0.9 per lot of my position.

Each hypothesis is a Python list of 13 numbers (names H_READS … H_TH, line 50): its fixed style, its belief over (O, t) as two means and a 2×2 covariance, and its log-weight.

3.4 Learning from an opponent quote: the Kalman filter (_see, lines 531–553)

Under a hypothesis, the opponent's quote is a straight line in (O, t) plus a little noise (SIG_Q = 0.5). For a Gaussian belief and a linear noisy reading, the **Kalman filter** is the exact Bayes update: new mean = old mean + gain × (quote − predicted quote), and the variance shrinks.

Example: an honest naive hypothesis in round 1 with belief O ~ N(35, 8^2). It predicts the quote 105.0. The opponent quotes 112.87, a surprise of 7.87. The gain is 1.262, so a pure Kalman step gives O ≈ 44.9 with sd 0.6: one honest quote almost reveals the hand.

We also allow a 10% chance that a quote is unexplained (EPS_Q). Here the quote is plausible, so it counts as explained with probability 0.929, and the belief becomes O ≈ 44.2 with sd 3.4. The hypothesis' log-weight changes by log(0.0281) = −3.57; hypotheses that predicted the quote better lose less weight.

3.5 Learning from an answer to my quote (_see, lines 554–590)

When I quote p and an opponent with take threshold th **passes**, its fair value must be between p − th and p + th. When it **buys**, its fair value is above p + th; when it **sells**, below p − th. That is an interval, not a number, so the update uses the mean and variance of a normal distribution cut to that interval (a **truncated normal**), followed by the same Kalman step.

Example (same hypothesis, I quote 100): before the answer, their fair value is believed to be 105.0 ± 6.4.

- They **pass** (fair value in 98..102): this was only 18% likely. O moves from 35 to 29.6 and its sd from 8 to 3.6.
- They **buy** (fair value above 102): 68% likely. O moves to 39.0, sd 5.7.

3.6 Weights and pruning (lines 553, 583, 427)

Each hypothesis' log-weight adds log(the probability it gave to what happened). This is Bayes' rule in log form. After every event, hypotheses more than 6 log-units behind the best (e^6 = 403 times less likely) are dropped (PRUNE), and the best is reset to 0 so the numbers never underflow.

3.7 Choosing a price (_quote, line 873)

- **1. Their answer model.** Under each of the 8 likeliest hypotheses, the opponent's fair value when answering me is normal with mean f and sd sf.
- **2. Immediate expected profit** of quoting p (function imm): they buy nb lots with probability Φ((f − p − th) / sf), and I sell at p + 2. But they buy exactly when their hand is high, which is when V is high, so the formula subtracts β × c × φ(z), the **adverse-selection** term (c = how much O moves with their fair value).
- **3. Stage 1:** try their fair value ± 2, 2.5, 3, 3.5, 4.5 and 6 and keep the price with the best immediate profit.
- **4. Stage 2:** for a short list of candidates (that price, their fair value, ± 3.05, ± 6.05, and extreme prices at the position limit), add a play-out of the rest of the deal (function _roll) and score the whole deal with the utility. Keep the best.
- **5. Nudge:** final quote = 0.7 × best + 0.3 × safe quote.

3.8 The utility (util, line 61)

util(x) = x / √(1 + |x| / 60). Small amounts count fully (util(10) = 9.3); large amounts are shrunk like a square root (util(100) = 61.2, util(400) = 144.5). The score uses sign(p)√|p| per deal pair, so we choose actions by the same shape: a sure +30 is worth more to us than a coin flip between 0 and +60, because util(30) = 24.5 is more than (0 + util(60)) / 2 = 21.2.

3.9 The swap (_move, line 1037)

In round 5, before the last two quotes, let pos be the position we hold now and fpos the position we expect to end with (from play-outs of round 5). matias uses x = 0.3 × fpos + 0.7 × pos. If x > 0 (we will be long), swapping our lowest card c for a random stack card (worth mstack on average) changes our PnL by about x × (mstack − c); if x < 0 we throw our highest card instead. We swap whenever that expected gain is positive. Example: pos = +5 and fpos = +8, so x = 0.3 × 8 + 0.7 × 5 = +5.9; lowest card 2, mstack 7: gain 5.9 × 5 = 29.5, which is positive, so we swap. Why the mix: the play-out tends to predict more round-5 trading than really happens, so leaning on the real position made the decision steadier (50/50 in pamplona gained +0.40 live; 70/30 was a little better again in local tests).

Your notes

4. Header and constants (lines 1–50)

This chapter covers the top of the file: the required header, our version note, the imports and every constant.

Lines 1–9 · The required header, our version note and the imports

```
1  # Team Name: Alpha Rodeo
2  # Members: Jaiv Patel(26075057)
3  # Mentor: Arpit Gupta
4  # Version: candidate tt01_plan8w3, parent pamplona, idea: NPLAN 8 + swap blend weight 0.3 (both
       positive locally on pamplona)
5
6
7
8  import heapq
9  import math
```

L1–3 The three lines every submission must start with, exactly as the rules give them: team name, member, mentor.

L4 Our version note for this upload. tt01 was its test name; in our ledger it is matias (v15), with its parent pamplona and the two changes (plan with 8 hypotheses; swap position 0.3 expected + 0.7 current). Python ignores comments.

L5–7 Empty. Our working copies also have a description of the whole bot here (34 lines); it was taken out before the upload, so the line numbers in this guide are those of the uploaded file.

L8–9 Only `heapq` (to pick the 8 heaviest hypotheses quickly) and `math` (sqrt, exp, log, erf). Both are on the allowed list. There is no file or network access anywhere in the bot.

> **SAY IT LIKE THIS**
> "Lines 1–3 are the required header, line 4 is our version note, and then come the two imports, both from the allowed list."

Lines 11–14 · Basic numbers

```
11 MAXP = 195.0
12 SQ2 = math.sqrt(2.0)
13 ISQ2PI = 1.0 / math.sqrt(2.0 * math.pi)
14 INF = float("inf")
```

L11 MAXP = 195 = 13 × 15, the highest possible V. The engine clamps every quote to 0..195.

L12 √2, used in Phi.

L13 1 / √(2π), the constant in the normal density.

L14 Infinity, used for one-sided intervals such as "their fair value is above 102".

Lines 16–29 · Opponent styles and their prior probabilities

```
16 RAW, CLAMP, SANE, CUR, BLD = 0, 1, 2, 3, 4
17 P_NAIVE = 0.3
18 P_BLEND = 0.12      # prior mass of blend quoters (the rest scaled by 1 - P_BLEND)
19 BLEND_LAM = ((1.0, 0.5), (0.7, 0.2), (0.4, 0.15), (0.2, 0.15))
20 BLEND_SK = ((0.0, 0.5), (0.9, 0.5))   # their quote skew per lot of MY position
21 W_MEAN, W_SD = 0.45, 0.25
22 BLEND_TAU = ((0.0, 0.3), (1.5, 0.4), (3.0, 0.3))
23 CARRY = 0.82      # how much a board card moves a carried fair, per point off 7
24 RULE_PRIOR = {RAW: 0.55, CLAMP: 0.2, SANE: 0.08, CUR: 0.17}
25 LAM_PRIOR = ((1.0, 0.45), (0.7, 0.12), (0.4, 0.08), (0.0, 0.35))
26 TH_PRIOR = ((2.0, 0.75), (7.0, 0.25))   # their take threshold: |fair - my price| (2 values for speed)
27 TAU_PRIOR = ((0.0, 0.6), (1.5, 0.25), (3.0, 0.15))      # only for honest quoters
28 KAP_PRIOR = ((0.0, 0.8), (3.9, 0.2))
29 T_MEAN, T_SD = 0.6, 0.3
```

L16 Five rule codes: RAW, CLAMP, SANE, CUR (the readers) and BLD (blend quoters). Small integers are faster to compare than strings.

L17 P_NAIVE = 0.3: prior chance that the opponent does not read my quotes at all.

L18 P_BLEND = 0.12: prior chance that it is a blend quoter. All other families share the remaining 0.88.

L19 BLEND_LAM: for blend quoters, how much of their hand shows in their own value: 1.0 (prob. 0.5), 0.7 (0.2), 0.4 (0.15), 0.2 (0.15).

L20 BLEND_SK: their quote moves 0 or 0.9 per lot of MY position (an inventory skew), 50/50.

L21 W_MEAN, W_SD: prior for the blend weight W that a blend quoter puts on my last quote: mean 0.45, sd 0.25.

L22 BLEND_TAU: for blend quoters, how much their value moves per lot I buy from them: 0, 1.5 or 3.

L23 CARRY = 0.82: when a blend quoter carries my old quote forward, each new board card moves it by 0.82 per point away from 7. A shown card changes the value of the stock by a bit less than its distance from 7, because the average of the cards still unknown moves the other way: about 1 − 9/46 = 0.80 in round 2 and 0.86 in round 5.

L24 RULE_PRIOR: among readers, RAW 55%, CLAMP 20%, SANE 8%, CUR 17%.

L25 LAM_PRIOR: how honest their quotes are: lam 1 (45%), 0.7 (12%), 0.4 (8%) or 0 (35%: quotes say nothing about the hand, as with strategic quoters and "hiders").

L26 TH_PRIOR: their take threshold: they trade on 2 points of edge (75%) or only on 7 (25%). Many strong teams only trade on large edges.

L27 TAU_PRIOR: 0 (60%), 1.5 (25%), 3 (15%); used only for lam = 1 quoters.

L28 KAP_PRIOR: 80% "does not expect me to swap" (0); 20% "moves its value by 3.9 in the direction my position says I will swap".

L29 T_MEAN, T_SD: prior trust of a reader in my quotes: mean 0.6, sd 0.3.

WHY / EVIDENCE

TH_PRIOR came in with helsinki (v06): modelling opponents who need 7 points of edge was worth +0.86 on our local field model. Taking the thresholds out again (test z02) lost −0.45 live, and moving the prior to 60/40 (test mm03, today 18:10) lost −0.53 ± 0.32 live, so 75/25 stays.

Lines 31–48 · Noise, speed and decision settings

```
31  SIG_Q = 0.5       # quote model noise
32  EPS_Q = 0.1       # chance a quote is unexplained
33  U_Q = 0.02        # density of an unexplained quote
34  SIG_R = 1.0       # response threshold noise
35  EPS_R = 0.06      # chance a response is unexplained
36  PRUNE = 6.0       # drop hypotheses this many log-units behind the best
37  SWAP_TH = (15.0, 12.0, 9.0, 6.0, 0.0)   # by round: waiting has option value
38  DELTAS = (2.0, 2.5, 3.0, 3.5, 4.5, 6.0)
39  NPLAN = 8         # hypotheses used in play-outs
40  P_EXT = 2.0       # confidence in a raw reader needed to PAY for an extreme quote
41  EX_TOL = 0.02      # exact-reader test: every quote within this of the house reader formula
42  P_PROBE = 2.0    # confidence in any reader needed to pay for a probe (implied 63 / 7)
43  LAM_PROBE = 0.4   # ... and only if its quotes are informative (strategic quoters are not)
44  RISK = 60.0       # utility curvature
45  SAFE_D = 1.3      # safe quote: this far from their last quote, towards my value
46  SAFE_W = 0.3      # share of the safe quote in my quote
47  OPP_SWAP_K = 0.6  # share of the opponent's coming swap priced into my value
48
```

L31 SIG_Q = 0.5: how far (sd) a quote can be from what a correct hypothesis predicts.

L32–33 EPS_Q = 0.1 and U_Q = 0.02: 10% of quotes may be "unexplained"; such a quote gets a flat probability density of 0.02 (as if spread evenly over about 50 points). One odd quote then cannot destroy a good hypothesis.

L34 SIG_R = 1.0: noise on the take threshold when we read answers.

L35 EPS_R = 0.06: 6% of answers may be random (each of buy, pass and sell then has 2%).

L36 PRUNE = 6.0: drop hypotheses 6 log-units (403 times) less likely than the best. It was 12 in bogota; 6 makes calls about 40% faster and changed only about 5% of decisions in our check (local score +0.014).

L37 SWAP_TH: the gain needed to swap, by round: 15, 12, 9, 6, 0. We only swap in round 5, so only the last value is used, and it is 0: we swap whenever the expected gain is positive. manila (v13) changed it

from marseille's 4 to 0, and pamplona keeps it.

L38 DELTAS: how far from their fair value the stage-1 price candidates are: 2, 2.5, 3, 3.5, 4.5, 6.

L39 NPLAN = 8: number of most likely hypotheses used in decisions (a speed versus accuracy choice). It was 6 until matias; 8 was better in all six local runs, and the slowest call is still under 10 ms.

L40 P_EXT = 2.0: the confidence needed to PAY for an extreme quote. A probability is never above 1, so 2.0 switches paid extremes off.

L41 EX_TOL = 0.02: the house-reader test: every quote must be within 0.02 of the house formula.

L42 P_PROBE = 2.0: the same trick: paid probes are off.

L43 LAM_PROBE = 0.4: for probes (now off), the reader would also have to show its hand in its quotes.

L44 RISK = 60: the curvature of the utility (section 3.8).

L45 SAFE_D = 1.3: the safe quote is 1.3 from their last quote, towards my value, inside their ±2 no-trade band.

L46 SAFE_W = 0.3: the nudge: 30% safe quote, 70% planner.

L47 OPP_SWAP_K = 0.6: belongs to _opp_swap (line 646), which is no longer called (since palermo).

WHY / EVIDENCE

The swap bar: lowering it improved the local score step by step (average of two seed pairs: bar 6 −0.060, 4 0, 2.5 +0.026, 1 +0.054, 0 +0.083). Live, manila (bar 0) beat marseille (bar 4) by +0.10 ± 0.05 on the same 37 opponents. Round 5 is the last chance to swap, so waiting has no value left.

WHY / EVIDENCE

Paid probes were switched off in stockholm (v09): in a local test one probe quote of 146.18 cost 100 points in a single deal, and turning probes off was neutral to positive. RISK 30 and 120 were tested: 120 lost (−0.08), 30 was not clearly better.

Lines 49–50 · Hypothesis fields

```
49  # hypothesis fields
50  H_READS, H_RULE, H_LAM, H_TAU, H_KAP, H_MO, H_MT, H_P00, H_P01, H_P11, H_LW, H_SK, H_TH = range(13)
```

L50 A hypothesis is a list of 13 numbers; these names are the positions in the list:

- H_READS (does it read my quotes?), H_RULE (RAW, CLAMP, SANE, CUR or BLD), H_LAM (honesty of its quotes), H_TAU (move per lot I buy), H_KAP (swap expectation), H_SK (blend skew), H_TH (take threshold). These are fixed for the hypothesis.
- H_MO and H_MT: the belief means of O and t. H_P00, H_P01, H_P11: the belief covariance (variance of O, covariance of O and t, variance of t). These are updated by learning.
- H_LW: the log-weight.
- Lists with index constants are faster in Python than objects with attributes, which matters for the time limit.

Your notes

5. Maths helpers (lines 53–170)

Small functions used everywhere: the normal curve, the utility, the opponent's value formula, and the Kalman update.

Lines 53–58 · The normal curve

FUNCTION CARD	`Phi(x) · line 53`
DOES	The standard normal cumulative probability: the chance that a standard normal value is at most x.
RETURNS	A number between 0 and 1.
WHY IT MATTERS	Turns "how many standard deviations away" into a probability: the chance the opponent buys or sells at our price, and how likely an answer was.
CALLED BY	_see (answer likelihood), imm inside _quote.

FUNCTION CARD	`pdf(x) · line 57`
DOES	The standard normal density at x.
RETURNS	A number of 0 or more.
WHY IT MATTERS	Needed for the mean shift of a normal cut to an interval (learning from answers) and for the adverse-selection term of a quote.
CALLED BY	_see, imm inside _quote.

```
53  def Phi(x):
54      return 0.5 * (1.0 + math.erf(x / SQ2))
55
56
57  def pdf(x):
58      return math.exp(-0.5 * x * x) * ISQ2PI
```

L53–54 Phi(x): the standard normal cumulative probability P(Z ≤ x), computed with math.erf.

L57–58 pdf(x): the standard normal density φ(x) = $e^{-x^2/2}$ / √(2π).

Lines 61–63 · The utility

FUNCTION CARD	`util(x) · line 61`
DOES	A concave utility: about x for small amounts and about √(60·|x|) for large ones, with the same sign as x.
RETURNS	A number with the sign of x.
WHY IT MATTERS	Makes the bot judge a whole deal the way the score does (square root per deal pair), so it prefers steady profit to gambles.
CALLED BY	_quote (stage 2) and _respond, on the whole deal's profit.

```
61  def util(x):
62      """Concave utility: linear for small PnL, square-root-like for large."""
63      return x / math.sqrt(1.0 + abs(x) / RISK)
```

L63 util(x) = x / √(1 + |x| / RISK). Close to x for small amounts, square-root-like for big ones, and symmetric: util(−x) = −util(x). See section 3.8.

SAY IT LIKE THIS
"We maximise a utility with the same shape as the score: small profits count fully, big ones only like a square root, so the bot prefers steady wins to gambles."

Lines 66–82 · The opponent's value as a straight line

FUNCTION CARD	`coefs(st, reads, rule, q, L) · line 66`
DOES	Writes one opponent style's fair value as a straight line in its hand sum O and its trust t: fair = A0 + A1·t + (B0 + B1·t)·O, with or without reading our quote q (capped at L for CLAMP readers).
RETURNS	A tuple (A0, A1, B0, B1).
WHY IT MATTERS	The core model of "what the opponent thinks the stock is worth". Because it is linear in O and t, the Kalman filter can update the beliefs exactly, and every learning and planning step relies on it.
CALLED BY	_see, _bel, _exact_reader, resp_dist_raw inside _quote.

```
66  def coefs(st, reads, rule, q, L):
67      """Opponent fair = A0 + A1*t + (B0 + B1*t)*O, before tau / kappa terms.
68
69      st: (bv, hb, npk, pk, N, C0, mo) public state; q: my quote it reads (None
70      for none); L: the clamped implied hand, or None.
71      """
72      bv, hb, npk, pk, N, C0, mo = st
73      cm0 = 5 - npk + hb
74      A0 = bv + pk + cm0 * C0 / N
75      B0 = 1 - cm0 / N
76      if not reads or q is None:
77          return A0, 0.0, B0, 0.0
78      if L is None:
79          c1 = 10 - npk + hb
80          return A0, (q - bv - pk) - c1 * C0 / N, B0, c1 / N
81      c1 = 5 - npk
82      return A0, (L - pk) - c1 * C0 / N, B0, c1 / N
```

st holds the public state as the opponent saw it (built by _state, line 434): bv = board sum, hb = hidden board cards, npk = number of their peeks, pk = estimated sum of my peeked cards, N = cards unknown to them, C0 = total of the cards they have not seen (still including their own hand), mo = the average unknown card if their hand is average.

L72 Unpack st.

L73 cm0 = the cards they cannot see that count in V, apart from their own hand: my unpeeked cards (5 − npk) plus the hidden board (hb).

L74 A0 = visible board + my peeked cards + cm0 cards at the average unknown value C0 / N (the O part of that average goes into B0).

L75 B0 = 1 − cm0 / N: their value rises by B0 per point of their own hand. It is below 1 because a high hand makes the unknown cards lower on average.

L76–77 No reading (naive, or I have not quoted yet): fair = A0 + B0·O.

L78–80 A raw reader adds t × [q − bv − pk − c1 × (C0 − O) / N] with c1 = 10 − npk + hb: my quote minus what a quote from an average hand would be. Written as a straight line: A1 = (q − bv − pk) − c1·C0 / N and B1 = c1 / N.

L81–82 A reader with a clamped implied hand L (5 or 65) adds t × (L − pk − (5 − npk) × the average unknown card): the clamped hand minus my average hand.

SAY IT LIKE THIS
"Every opponent style we model values the stock as a straight line in its hand sum O and its trust t. That is what lets us track it with a Kalman filter."

Lines 85–86 · implied

FUNCTION CARD	`implied(st, q) · line 85`
DOES	The hand sum of ours that quote q implies to an opponent with an average hand.
RETURNS	A number (a hand sum).
WHY IT MATTERS	Lets CLAMP and SANE readers spot impossible quotes (outside 5–65 or 12–58), and gives the "hand 65" and "hand 5" prices at the position limit.
CALLED BY	pick_q, _bel, _roll, _quote.

```
85  def implied(st, q):
86      return q - st[0] - (5 + st[1]) * st[6]
```

L86 The hand of mine that quote q implies, for an average opponent hand: q − board − (5 + hidden board cards) × the average unknown card. The CLAMP and SANE readers use it to spot impossible quotes.

Lines 89–105 · pick_q: which of my quotes a reader uses

FUNCTION CARD	`pick_q(st, rule, mq) · line 89`
DOES	Decides which of our past quotes a reader with a given rule is reading now: the newest one, or for SANE the newest plausible one; for CLAMP also the cap.
RETURNS	A pair (quote or None, cap or None).
WHY IT MATTERS	Different readers read different quotes; the right one has to go into coefs.
CALLED BY	_see, _plan_setup, _move.

```
89  def pick_q(st, rule, mq):
90      """Which of my quotes (newest last) a reader of this rule uses, and clamp."""
91      if not mq:
92          return None, None
93      if rule == SANE:
94          for x in reversed(mq):
95              if 12 <= implied(st, x) <= 58:
96                  return x, None
97          return None, None
98      q = mq[-1]
99      if rule == CLAMP:
100         imp = implied(st, q)
101         if imp > 65:
102             return q, 65.0
103         if imp < 5:
104             return q, 5.0
105     return q, None
```

L91–92 No quote of mine yet: nothing to read.

L93–97 SANE: the newest quote whose implied hand is between 12 and 58; if there is none, nothing.

L98 Every other rule reads my newest quote.

L99–104 CLAMP: if the quote implies more than 65 or less than 5, also return the clamp value.

L105 RAW and CUR: the newest quote, no clamp.

Lines 108–120 · ekf: one Kalman step (the readable version)

FUNCTION CARD	ekf(h, Ho, Ht, d, S, Vy) · line 108
DOES	One Kalman step for a hypothesis' belief over (O, t): moves the means by gain × surprise and shrinks the covariance.
RETURNS	A tuple (mean O, mean t, P00, P01, P11); h itself is not changed.
WHY IT MATTERS	The readable reference version of the belief update; the bot runs the fused ekf_mix.
CALLED BY	Nothing at run time.

```
108 def ekf(h, Ho, Ht, d, S, Vy):
109     """Kalman step for hypothesis h: a reading with gradient (Ho, Ht) in (O, t)
110     and predictive variance S came out d above its prediction; Vy is what is
111     left of its variance afterwards (0 when it was seen exactly).
112     Returns the new (mean O, mean t, P00, P01, P11)."""
113     P00, P01, P11 = h[H_P00], h[H_P01], h[H_P11]
114     PHo = P00 * Ho + P01 * Ht
115     PHt = P01 * Ho + P11 * Ht
116     k0 = PHo / S
117     k1 = PHt / S
118     g = S - Vy
119     return (h[H_MO] + k0 * d, h[H_MT] + k1 * d,
120             P00 - g * k0 * k0, P01 - g * k0 * k1, P11 - g * k1 * k1)
```

The belief of a hypothesis is a Gaussian over (O, t) with means (H_MO, H_MT) and covariance P = [[P00, P01], [P01, P11]]. An observation y behaves like Ho·O + Ht·t + noise; S is its predicted variance and d is how far it came out above the prediction.

L113 Read the covariance.

L114–115 P·H: how the observation moves together with O and with t.

L116–117 The Kalman gains k = P·H / S: how much to move each mean per point of surprise.

L118 g = S − Vy: how much of the observation's uncertainty was removed. For a quote all of it (Vy = 0); for an answer only part of it, because an answer only gives an interval.

L119–120 New means = old means + k·d; new covariance = P − g·k·k^T.

Note: ekf and mix_into are no longer called. ekf_mix (line 141) does both in one step for speed; these two stay as the readable reference.

Lines 123–138 · mix_into: hedging against outliers

FUNCTION CARD	mix_into(h, r, new) · line 123
DOES	Blends the updated belief (weight r) with the old one (1 − r) into one Gaussian with the same mean and covariance, and keeps the matrix valid.
RETURNS	Nothing; changes h in place.
WHY IT MATTERS	Hedges against outliers: an observation that may be unexplained should not fully rewrite a belief.
CALLED BY	Nothing at run time (fused into ekf_mix).

```
123 def mix_into(h, r, new):
124     """Moment-match r*new + (1-r)*old into h."""
125     m0, m1, a, b, c = new
126     o0, o1 = h[H_MO], h[H_MT]
127     n0 = r * m0 + (1 - r) * o0
128     n1 = r * m1 + (1 - r) * o1
129     h[H_P00] = max(0.05, r * (a + m0 * m0) + (1 - r) * (h[H_P00] + o0 * o0) - n0 * n0)
130     h[H_P01] = r * (b + m0 * m1) + (1 - r) * (h[H_P01] + o0 * o1) - n0 * n1
131     h[H_P11] = max(1e-5, r * (c + m1 * m1) + (1 - r) * (h[H_P11] + o1 * o1) - n1 * n1)
132     h[H_MO] = n0
133     h[H_MT] = min(1.15, max(0.0, n1))
134     lim = math.sqrt(h[H_P00] * h[H_P11]) * 0.999
135     if h[H_P01] > lim:
136         h[H_P01] = lim
137     elif h[H_P01] < -lim:
```

```
138         h[H_P01] = -lim
```

After the update we are not sure the observation was "explained" (see EPS_Q). With probability r the new belief is right, with 1 − r the old one is. mix_into replaces this mixture of two Gaussians by one Gaussian with the same mean and covariance ("moment matching").

L127–128 The mixture means.

L129–131 The mixture covariance = $E[x \cdot x^T] - \text{mean} \cdot \text{mean}^T$. The floors 0.05 and 0.00001 keep the variances positive.

L132–133 Store the means; the trust t is kept between 0 and 1.15.

L134–138 Keep |cov(O, t)| below √(P00·P11), so the correlation stays below 1 and the matrix stays valid.

Lines 141–170 · ekf_mix: the same, fused (the one actually used)

FUNCTION CARD	`ekf_mix(h, r, Ho, Ht, d, S, Vy) · line 141`
DOES	ekf followed by mix_into, written out as one function.
RETURNS	Nothing; updates h's belief in place.
WHY IT MATTERS	The learning step run for every hypothesis on every event. Fusing it was part of marseille's 30–40% speed-up.
CALLED BY	_see.

```
141 def ekf_mix(h, r, Ho, Ht, d, S, Vy):
142     """ekf() followed by mix_into(h, r, ...), fused: the same arithmetic in the same order,
143     without building a tuple for every hypothesis and observation."""
144     P00, P01, P11 = h[H_P00], h[H_P01], h[H_P11]
145     PHo = P00 * Ho + P01 * Ht
146     PHt = P01 * Ho + P11 * Ht
147     k0 = PHo / S
148     k1 = PHt / S
149     g = S - Vy
150     o0, o1 = h[H_MO], h[H_MT]
151     m0 = o0 + k0 * d
152     m1 = o1 + k1 * d
153     a = P00 - g * k0 * k0
154     b = P01 - g * k0 * k1
155     c = P11 - g * k1 * k1
156     n0 = r * m0 + (1 - r) * o0
157     n1 = r * m1 + (1 - r) * o1
158     p00 = max(0.05, r * (a + m0 * m0) + (1 - r) * (P00 + o0 * o0) - n0 * n0)
159     p01 = r * (b + m0 * m1) + (1 - r) * (P01 + o0 * o1) - n0 * n1
160     p11 = max(1e-5, r * (c + m1 * m1) + (1 - r) * (P11 + o1 * o1) - n1 * n1)
161     h[H_P00] = p00
162     h[H_P11] = p11
163     h[H_MO] = n0
164     h[H_MT] = min(1.15, max(0.0, n1))
165     lim = math.sqrt(p00 * p11) * 0.999
166     if p01 > lim:
167         p01 = lim
168     elif p01 < -lim:
169         p01 = -lim
170     h[H_P01] = p01
```

The same arithmetic as ekf followed by mix_into, written out in one function so that Python does not build a tuple for every hypothesis and every observation. This was part of marseille's speed-up. Lines 144–155 are ekf; lines 156–170 are mix_into.

SAY IT LIKE THIS

"Each hypothesis keeps a Gaussian belief. Every observation moves it with a Kalman step, blended with the old belief by the chance that the observation was not an outlier."

Your notes

6. The house-reader attack (lines 173–381)

What this part is for

The arena had three house bots: naive, reader and sharp. The reader and the sharp value the stock with an exact formula that trusts my last quote with weight 0.8. Once we know their hand sum S exactly, we can predict every quote and answer they will make, and plan a sequence of quotes that steers their belief and then trades against it. Since 15:40 the house bots no longer count in the arena score, but this layer is harmless: it switches on only after two of their quotes fit the formula to the cent (no real team triggered it in 400 deals against 45 teams), and it switches off at the first answer that does not fit.

WHY / EVIDENCE
It was added in nairobi (v02): +7.4 against the house reader and +10.0 against the sharp, with exactly the same decisions as before against every real team.

Lines 173–187 · Constants of the attack

```
173 # ── exact house-reader attack ──────────────────────────────────────────
174 # The house reader and sharp quote fair = S + (1-T)*board + T*P + m*(5 + h - T*(10 + h)) with
175 # T = 0.8 and P = my last quote, rounded to 0.01. Once two of their quotes (one of them reading
176 # a quote of mine) give the same whole hand sum S, they are steered and harvested with an exact
177 # look-ahead (our merge line's attack layer). The first answer that differs from the model
178 # switches the attack off for the rest of the deal.
179 DECK_SUM = 364
180 RD_T = 0.8          # the house reader / sharp trust in my last quote
181 RD_TOL = 0.0051     # their quotes are rounded to 0.01
182 RD_NQ = 2           # consistent quotes needed, at least one of them reading a quote of mine
183 STEER_IN = 1.98     # inside their +-2 band: no trade, their belief moves
184 HARVEST_OUT = 3.02  # outside it: sharp's sizing gives 2 lots at edge >= 3
185 RESP_MARGIN = 0.2   # extra edge wanted when answering an identified reader
186 DEPTH = 4           # quotes of mine explored with every option
187 LIMIT = 10
```

L174–178 The house formula: fair = S + (1 − T)·board + T·P + m·(5 + h − T·(10 + h)), with T = 0.8, P = my last quote, m = the average unseen card and h = hidden board cards.

L179 DECK_SUM = 364, the sum of all 52 cards.

L180 RD_T = 0.8, their trust in my quote.

L181 RD_TOL = 0.0051: their quotes are rounded to 0.01, so a true fit is within half a cent.

L182 RD_NQ = 2: two consistent quotes needed, at least one of them reading a quote of mine.

L183 STEER_IN = 1.98: a quote 1.98 from their value is inside their ±2 band: no trade, but they read my quote and their belief moves.

L184 HARVEST_OUT = 3.02: just outside the band: the sharp then trades 2 lots.

L185 RESP_MARGIN = 0.2: when answering an identified reader, take only with 2.2 points of edge (a small safety margin).

L186 DEPTH = 4: how many of my future quotes the exact search explores with every option.

L187 LIMIT = 10: the position limit.

Lines 190–196 · _lots: the sharp's trade size

FUNCTION CARD	`_lots(e) · line 190`
DOES	The house sharp bot's trade size for an edge e.
RETURNS	Signed lots: 0, ±1 or ±2.
WHY IT MATTERS	The exact attack must predict exactly how much the house bot trades at each price.
CALLED BY	_Reader.plan.

```
190 def _lots(e):
191     """sharp's sizing for a taker with edge e: signed lots (+ buy)."""
192     a = abs(e)
193     if a <= 2:
194         return 0
195     n = min(2, int(a - 2) + 1)
196     return n if e > 0 else -n
```

L192–196 For an edge e (their value − my price): no trade within 2; 1 lot when 2 < |e| < 3; 2 lots from 3 up. The sign says buy (+) or sell (−).

Lines 199–210 · _Reader: its memory

FUNCTION CARD	`class _Reader · __init__() · line 199`
DOES	The exact attack on the house reader and sharp bots. __init__ sets up its memory: the candidate hand sum S, counts of consistent quotes, the peeked and off flags, and the pending check.
RETURNS	Nothing.
WHY IT MATTERS	The attack stays off until the evidence is overwhelming; this state collects that evidence over the deal.
CALLED BY	Bot.__init__ creates one per deal.

```
199 class _Reader:
200     """Identifies an exact house reader and plans quotes against it."""
201 
202     def __init__(self):
203         self.S = None        # hand sum of the current run of consistent quotes
204         self.nq = 0          # consistent quotes in that run
205         self.nread = 0       # ... of them made after a quote of mine
206         self.done = 0        # their quotes processed
207         self.last_round = 0
208         self.peeked = False
209         self.off = False     # an answer did not fit the model
210         self.check = None    # (tape index, expected sign of their qty, my position then)
```

L203 S: their hand sum in the current run of consistent quotes.

L204–205 nq: how many quotes are in the run; nread: how many of them read a quote of mine.

L206 done: how many of their quotes were already processed.

L207 last_round: the round of the last processed quote (to notice their swaps).

L208 peeked: they peeked (their formula then changes), so give up.

L209 off: an answer did not fit; the attack is off for the rest of the deal.

L210 check: the answer we expect to my last planned quote, so we can verify the model.

Lines 212–224 · invert: their hand sum from one quote

FUNCTION CARD	`invert(F, B, h, D, N, P) · line 213`
DOES	Solves the house formula backwards: from one opponent quote F, finds the whole-number hand sum S that produces it exactly.
RETURNS	S (a whole number from 5 to 65), or None if nothing fits within 0.0051.
WHY IT MATTERS	An exact fit both identifies a house bot and reveals its hand completely.
CALLED BY	observe.

```
212     @staticmethod
213     def invert(F, B, h, D, N, P):
214         U0 = DECK_SUM - B - D
215         if P is None:
216             c0, c1 = B, 5 + h
217         else:
218             c0, c1 = (1 - RD_T) * B + RD_T * P, 5 + h - RD_T * (10 + h)
```

```
        a = 1 - c1 / N
        S = round((F - c0 - c1 * U0 / N) / a)
        if not 5 <= S <= 65:
            return None
        pred = S + c0 + c1 * (U0 - S) / N
        return S if abs(pred - F) <= RD_TOL else None
```

Solve the house formula for S. F = their quote, B = board sum, h = hidden cards, D = discard sum, N = cards unknown to them, P = my last quote (None if I have not quoted).

L214 U0 = the sum of all cards not on the board and not discarded (their hand + my hand + hidden board + stack).

L215–218 The formula written as F = S + c0 + c1·(U0 − S) / N. Without a quote of mine c0 = board and c1 = 5 + h; with one, the 0.8 trust terms come in.

L219–220 Rearranged: S = (F − c0 − c1·U0 / N) / (1 − c1 / N), rounded to a whole number (a hand sum is an integer).

L221–222 A 5-card hand sum lies between 5 and 65; otherwise this is not the formula.

L223–224 Recompute the quote from that S and accept it only if it matches to 0.0051.

Lines 226–249 · observe: is this quote consistent with the earlier ones?

FUNCTION CARD	`observe(F, rnd, obs, P) · line 226`
DOES	Checks one new opponent quote against the current run of consistent quotes (the same S, or a consistent change after the opponent's swap).
RETURNS	Nothing; updates S, nq and nread.
WHY IT MATTERS	Two consistent quotes, one of them reading our quote, are the trigger for the attack.
CALLED BY	sync.

```
    def observe(self, F, rnd, obs, P):
        self.done += 1
        if self.peeked:
            return
        B, h = sum(obs.board[:rnd - 1]), 5 - (rnd - 1)
        ds = [(r, w, k) for r, w, k in obs.discards if r <= rnd]
        D, N = sum(k for _, _, k in ds), 47 - (rnd - 1) - len(ds)
        olds = [k for r, w, k in ds if w == "opp" and r > self.last_round]
        self.last_round = rnd
        S = self.invert(F, B, h, D, N, P)
        if S is not None and self.S is not None:
            if olds:
                lo = self.S - olds[0]
                ok = lo + 1 <= S <= lo + 13
            else:
                ok = S == self.S
            if ok:
                self.S = S
                self.nq += 1
                self.nread += P is not None
                return
        self.S = S
        self.nq = 1 if S is not None else 0
        self.nread = 1 if (S is not None and P is not None) else 0
```

L227 Count it.

L228–229 If they peeked, stop: their formula now includes my peeked card.

L230–232 The public numbers in that round: board sum B, hidden board h, the discards, D and N.

L233 Their discards since the last quote I processed (a swap changes S).

L235 S from this quote alone.

L236–246 If both this quote and the run give a hand sum, they are consistent when they are equal (no swap), or, after they threw card k, when the new sum is between old − k + 1 and old − k + 13 (the new card is 1 to 13). Then the run grows.

L247–249 Otherwise a new run starts with this quote.

Lines 251–271 · sync: feed new quotes and verify predictions

FUNCTION CARD	`sync(obs, pending=None) · line 251`
DOES	Feeds every new opponent quote to observe, notes peeks, and checks that the opponent answered our last planned quote as predicted (if not, the attack is off for the deal).
RETURNS	Nothing.
WHY IT MATTERS	Keeps the attack's picture current and switches it off at the first surprise, which makes it safe against real teams.
CALLED BY	Bot.quote, Bot.respond.

```
251     def sync(self, obs, pending=None):
252         if any(w == "opp" for _, w in obs.peeks):
253             self.peeked = True
254         quotes, P = [], None
255         for r, who, px, q in obs.tape:
256             if who == "me":
257                 P = px
258             else:
259                 quotes.append((px, r, P))
260         if pending is not None:
261             quotes.append((pending, obs.round, P))
262         for px, r, p in quotes[self.done:]:
263             self.observe(px, r, obs, p)
264         if self.check is not None:
265             idx, want, pos_before = self.check
266             if idx < len(obs.tape):
267                 q = obs.tape[idx][3]
268                 blocked = (want > 0 and pos_before <= -LIMIT) or (want < 0 and pos_before >= LIMIT)
269                 if not blocked and ((q > 0) != (want > 0) or (q < 0) != (want < 0)):
270                     self.off = True
271                 self.check = None
```

L252–253 Any peek by the opponent sets peeked.

L254–259 Walk the tape, remembering my latest quote P; each opponent quote is stored with the quote of mine it would read.

L260–261 When I am answering, their current quote is not on the tape yet, so add it.

L262–263 Process only the new ones.

L264–271 If I made a planned quote, compare their actual answer with the predicted direction. A mismatch that the position limit does not explain switches the attack off.

Lines 273–276 · active

FUNCTION CARD	`active(obs) · line 273`
DOES	Decides whether the exact attack may act now.
RETURNS	True or False.
WHY IT MATTERS	The gate: only an identified, unpeeked, unswapped house bot is attacked. Against real teams it stays False.
CALLED BY	Bot.quote, Bot.respond.

```
273     def active(self, obs):
274         if self.off or self.peeked or self.S is None or self.nq < RD_NQ or self.nread < 1:
275             return False
276         return not any(w == "opp" and r > self.last_round for r, w, k in obs.discards)
```

L274 On only if: not off, no peek, S known, at least 2 consistent quotes, at least 1 of them reading my quote.

L276 And they have not swapped since their last quote (S would be unknown again).

Lines 278–282 · value: V when S is known

FUNCTION CARD	`value(obs) · line 278`
DOES	The stock's expected value once the opponent's hand sum S is known.
RETURNS	A number.
WHY IT MATTERS	The attack plans against the true value, not an estimate.
CALLED BY	_Reader.quote, _Reader.respond.

```
278     def value(self, obs):
279         n = sum(obs.unseen)
280         tot = sum((r + 1) * c for r, c in enumerate(obs.unseen))
281         rest = (tot - self.S) / max(n - 5, 1)
282         return sum(obs.hand) + sum(obs.board) + self.S + (5 - len(obs.board)) * rest
```

L279–281 The unseen pool contains their 5 cards (sum S); the rest average (total − S) / (count − 5).

L282 V = my hand + board + S + (hidden board cards) × that average.

Lines 284–293 · their_base: their value for a given S and my last quote P

FUNCTION CARD	`their_base(obs, S, P) · line 285`
DOES	The house bot's fair value for hand sum S, given our last quote P (or None).
RETURNS	A number.
WHY IT MATTERS	Predicts the house bot's quotes and answers exactly.
CALLED BY	_Reader.quote.

```
284     @staticmethod
285     def their_base(obs, S, P):
286         B = sum(obs.board)
287         h = 5 - len(obs.board)
288         D = sum(k for _, _, k in obs.discards)
289         N = 47 - len(obs.board) - len(obs.discards)
290         m = (DECK_SUM - S - B - D) / N
291         if P is None:
292             return S + B + (5 + h) * m
293         return S + (1 - RD_T) * B + RD_T * P + m * (5 + h - RD_T * (10 + h))
```

L286–290 Board sum, hidden cards, discards, N, and the average unknown card m from their side.

L291–292 Without a quote of mine: S + board + (5 + h)·m.

L293 With one: the 0.8-trust formula.

Lines 295–306 · events_left: who quotes next, until the end of the deal

FUNCTION CARD	`events_left(obs) · line 296`
DOES	Lists the remaining quotes of the deal, starting with ours now.
RETURNS	A list of (who, round), who being "me" or "opp".
WHY IT MATTERS	The look-ahead needs the exact order of the remaining moves.
CALLED BY	_Reader.quote.

```
295     @staticmethod
296     def events_left(obs):
297         """Remaining quote events from this quote of mine on: ('me'/'opp', round)."""
298         first = not any(r == obs.round for r, _, _, _ in obs.tape)
299         ev = []
300         for r in range(obs.round, 6):
301             mine_first = first if (r - obs.round) % 2 == 0 else not first
302             pair = ["me", "opp"] if mine_first else ["opp", "me"]
303             if r == obs.round:
```

```
                pair = ["me", "opp"] if first else ["me"]
            ev += [(w, r) for w in pair]
        return ev
```

L298 Am I the first maker in this round (no tape entry this round yet)?

L300–305 List the remaining quotes as ("me" or "opp", round): what is left of this round, then two per round, alternating who starts.

Lines 308–363 · plan: exact look-ahead against the house bot

FUNCTION CARD	`plan(events, pos, Pl, Fnone, A, V) · line 308`
DOES	Exact depth-first search over the remaining quotes. At each of our quotes it tries our true value, steering (±1.98), harvesting (±3.02) and the extremes 195 and 0, simulates the house bot's exact reaction, and adds our answers to its quotes. Results are memoised.
RETURNS	A pair (best total profit, the best quote to make now).
WHY IT MATTERS	Turns full knowledge of a house bot into the most profitable sequence of quotes: +7.4 against the house reader and +10.0 against sharp in nairobi's live test.
CALLED BY	_Reader.quote.

```
    def plan(self, events, pos, Pl, Fnone, A, V):
        """Best first quote over the remaining events (exact reader model)."""
        L = LIMIT
        t = RD_T

        def F(P):
            return Fnone if P is None else A + t * P

        memo = {}

        def dfs(i, pos, Pl, depth):
            if i >= len(events):
                return 0.0, None
            key = (i, pos, Pl, depth)
            got = memo.get(key)
            if got is not None:
                return got
            got = memo[key] = step(i, pos, Pl, depth)
            return got

        def step(i, pos, Pl, depth):
            who, r = events[i]
            if who == "opp":
                Q = F(Pl)
                g = 0.0
                if V - Q - 2 > RESP_MARGIN and pos < L:
                    n = min(2, L - pos)
                    g, pos = n * (V - Q - 2), pos + n
                elif Q - 2 - V > RESP_MARGIN and pos > -L:
                    n = min(2, L + pos)
                    g, pos = n * (Q - 2 - V), pos - n
                return g + dfs(i + 1, pos, Pl, depth)[0], None
            Fl = F(Pl)
            opts = [V]
            if depth > 0:
                opts += [Fl + STEER_IN, Fl - STEER_IN, Fl + HARVEST_OUT, Fl - HARVEST_OUT, 195.0, 0.0]
            best, bp = -1e18, V
            for P in opts:
                P = round(min(195.0, max(0.0, P)), 2)
                n = _lots(Fl - P)                    # + they buy from me
                if n > 0:
                    n = min(n, pos + L)
                elif n < 0:
                    n = max(n, pos - L)
                if n > 0:
                    g = n * (P + 2 - V)
                elif n < 0:
                    g = -n * (V - P + 2)
                else:
                    g = 0.0
                v = g + dfs(i + 1, pos - n, P, depth - 1)[0]
```

```
359                 if v > best + 1e-9:
360                     best, bp = v, P
361             return best, bp
362
363         return dfs(0, pos, Pl, DEPTH if len(events) < 7 else DEPTH - 1)   # keeps round 2 fast
```

A depth-first search over the remaining quotes, with memoisation (memo stores the result for each (event, position, my last quote, depth)).

L313–314 F(P): their value if my last quote is P (A + 0.8·P), or Fnone if I have not quoted.

L318–326 dfs with its cache.

L329–339 Their quote equals their value. I buy 2 if V is above their price + 2 + margin, sell 2 if it is below their price − 2 − margin (within limits).

L340–343 My quote: the options are my true value V (no steering) or, while depth remains, ±1.98 (steer), ±3.02 (harvest), 195 or 0 (extremes).

L345–360 For each option: their trade size by the sharp's rule, cut by the position limit; my profit at the true value V; plus the best continuation, in which they now read my new quote.

L363 Depth 4, or 3 when 7 or more events are left (keeps round 2 fast).

Lines 365–381 · quote and respond against an identified reader

FUNCTION CARD	`_Reader.quote(obs) · _Reader.respond(obs, price) · line 365`
DOES	quote runs plan and remembers which answer it expects; respond trades 2 lots when the edge against the known value beats 2.2.
RETURNS	quote: the price to quote. respond: +2, −2 or 0.
WHY IT MATTERS	The attack's actions, used instead of the normal planner while active() is True.
CALLED BY	Bot.quote and Bot.respond.

```
365     def quote(self, obs):
366         V = self.value(obs)
367         mine = [px for r, who, px, q in obs.tape if who == "me"]
368         Pl = mine[-1] if mine else None
369         Fnone = self.their_base(obs, self.S, None)
370         A = self.their_base(obs, self.S, 0.0)
371         _, P = self.plan(self.events_left(obs), obs.position, Pl, Fnone, A, V)
372         if abs(P - V) > 1e-9:
373             e = (Fnone if Pl is None else A + RD_T * Pl) - P
374             want = 0 if abs(e) <= 2 else (1 if e > 0 else -1)
375             self.check = (len(obs.tape), want, obs.position)
376         return P
377
378     def respond(self, obs, price):
379         edge = self.value(obs) - price
380         need = 2.0 + RESP_MARGIN
381         return 2 if edge > need else (-2 if edge < -need else 0)
```

L366–371 My value, my last quote, their value without and with my quote, then plan.

L372–375 If the chosen price is not plain V, remember which way we expect them to answer, to check it next time.

L379–381 Answering them: trade 2 lots when the edge beats 2.2.

SAY IT LIKE THIS

"The _Reader class is a special-purpose exact attack on the organisers' house bots. It only switches on when their quotes match the house formula to the cent, and it turns itself off at the first surprise, so against real teams it does nothing."

Your notes

7. The Bot: inference (lines 384–716)

How the bot turns what it sees into beliefs about the opponent.

Lines 384–391 · Creating the bot

FUNCTION CARD	`Bot.__init__() · line 385`
DOES	Creates an empty bot for one deal: no hypotheses yet, all counters at zero, seat unknown, and a fresh house-bot attack layer.
RETURNS	Nothing.
WHY IT MATTERS	The engine builds a new bot every deal, so nothing is carried over between deals (a rule of the event).
CALLED BY	The engine.

```
384 class Bot:
385     def __init__(self):
386         self.H = None        # hypotheses
387         self.done = 0        # tape entries consumed
388         self.ndisc = 0       # discards consumed
389         self.extra = -1      # tape slot of their quote consumed early
390         self.first = None    # do I quote first in odd rounds?
391         self.rd = _Reader()  # exact house-reader attack
```

The engine creates a fresh Bot for every deal, so nothing is kept between deals.

L386 H: the hypotheses (built at the first call, when the first obs is available).

L387 done: how many tape entries we have already learned from.

L388 ndisc: how many discards we have already processed.

L389 extra: the tape slot of their quote that we learned from early (while answering it).

L390 first: do I quote first in odd rounds (am I seat 0)? Learned at the first call.

L391 The house-reader attack layer.

Lines 393–425 · _init: the prior and the 216 hypotheses

FUNCTION CARD	`_init(obs) · line 394`
DOES	Builds the 216 hypotheses with their prior weights and starting beliefs: O about 35 ± 8, trust 0.6 ± 0.3 for readers, blend weight 0.45 ± 0.25 for blend quoters.
RETURNS	Nothing; sets self.H and self.obar.
WHY IT MATTERS	This is the prior: every opponent style starts with a weight, and learning moves weight to the styles that predict the opponent well.
CALLED BY	_sync, on the first call of the deal.

```
393     # ── inference ──────────────────────────────────────────
394     def _init(self, obs):
395         u = obs.unseen
396         U = sum(u)
397         S = sum((r + 1) * c for r, c in enumerate(u))
398         m = S / U
399         v = sum(c * (r + 1 - m) ** 2 for r, c in enumerate(u)) / U
400         mu = 5 * m
401         var = 5 * v * (U - 5) / (U - 1)
402         self.obar = mu
403         H = []
404         combos = [(False, RAW, P_NAIVE)] + [(True, r, (1 - P_NAIVE) * p) for r, p in RULE_PRIOR.items()]
405         for reads, rule, pr in combos:
406             for lam, pl in LAM_PRIOR:
407                 for tau, pt in TAU_PRIOR:
408                     if tau and lam < 1:
```

```
                        continue
                    if lam < 1:
                        pt = 1.0
                    for kap, pk in KAP_PRIOR:
                        for th, pth in TH_PRIOR:
                            lw = math.log((1 - P_BLEND) * pr * pl * pt * pk * pth)
                            H.append([reads, rule, lam, tau, kap, mu, T_MEAN if reads else 0.0,
                                      var, 0.0, T_SD * T_SD if reads else 1e-6, lw, 0.0, th])
        for lam, pl in BLEND_LAM:
            for tau, pt in BLEND_TAU:
                for kap, pk in KAP_PRIOR:
                    for sk, ps in BLEND_SK:
                        for th, pth in TH_PRIOR:
                            lw = math.log(P_BLEND * pl * pt * pk * ps * pth)
                            H.append([False, BLD, lam, tau, kap, mu, W_MEAN,
                                      var, 0.0, W_SD * W_SD, lw, sk, th])
        self.H = H
```

L395–398 From my unseen cards: count U, sum S, average m.

L399 The variance of one unseen card.

L400–401 Their hand sum O is 5 cards from that pool: mean 5m and variance 5v·(U − 5) / (U − 1) (cards drawn without replacement). At the start this is about 35 ± 8.

L402 obar = the prior mean of O, our "average hand".

L404 The families: naive (reads = False) and the four reader rules, with their prior probabilities.

L406–413 Every combination of lam, tau, kappa and threshold. tau is only allowed for lam = 1 (L408–409), and for lam < 1 the tau factor is 1 (L410–411).

L414 The log of the prior probability (the product of the factors), times 0.88 (= not a blend quoter).

L415–416 The new hypothesis: style fields; belief mean O = mu; t = 0.6 for readers (0 for naive); variance of O; no correlation; variance of t (tiny for naive); log-weight; skew 0; threshold.

L417–424 Blend quoters: every lam, tau, kappa, skew and threshold. For them "t" is the blend weight W (mean 0.45, sd 0.25).

L425 Store them: 120 + 96 = 216 hypotheses.

Lines 427–431 · _prune

FUNCTION CARD	`_prune() · line 427`
DOES	Drops hypotheses more than 6 log-units behind the best and shifts the weights so the best is 0.
RETURNS	Nothing.
WHY IT MATTERS	Keeps each call fast and the numbers stable.
CALLED BY	_see, after every event.

```
    def _prune(self):
        best = max(h[H_LW] for h in self.H)
        self.H = [h for h in self.H if h[H_LW] > best - PRUNE]
        for h in self.H:
            h[H_LW] -= best
```

L428 The best log-weight.

L429 Keep hypotheses within PRUNE (6) of the best.

L430–431 Shift so that the best is 0 (exp() then never underflows).

Lines 433–453 · _state: the public numbers as the opponent saw them in round r

FUNCTION CARD	`_state(obs, r, obar) · line 434`
DOES	The public numbers as the opponent saw them in round r.
RETURNS	A tuple: board sum, hidden board cards, their peeks, our peeked sum, cards unknown to them, total of those cards, average unknown card.
WHY IT MATTERS	The opponent's view of the deal, which is what coefs needs.
CALLED BY	_see, _exact_reader, _plan_setup, _move.

```
433     @staticmethod
434     def _state(obs, r, obar):
435         """Public state as the opponent saw it in round r."""
436         board = obs.board[:r - 1]
437         nb = len(board)
438         bv = sum(board)
439         nd = 0
440         ds = 0
441         for d in obs.discards:
442             if d[0] <= r:
443                 nd += 1
444                 ds += d[2]
445         npk = 0
446         for p in obs.peeks:
447             if p[1] == "opp" and p[0] <= r:
448                 npk += 1
449         npk = min(npk, 5)
450         pk = npk * sum(obs.hand) / 5.0
451         N = 47 - nb - nd - npk
452         C0 = 364 - bv - ds - pk
453         return (bv, 5 - nb, npk, pk, N, C0, (C0 - obar) / N)
```

L436–438 The board cards visible in round r (r − 1 of them), their count and sum.

L439–444 Swaps made up to round r: count nd and sum ds.

L445–449 How many times the opponent had peeked by round r (at most 5).

L450 pk: we do not know which of my cards they saw, so each peek counts as the average card of my hand.

L451 N = 47 − board − discards − peeks: the cards unknown to them (52, minus their own 5, minus what they have seen).

L452 C0 = 364 − board − discards − peeked: the total of the cards they have not seen, still including their own hand.

L453 Return (bv, hidden board cards, npk, pk, N, C0, the average unknown card for an average hand).

Lines 455–467 · _before: my position and Q before slot j

FUNCTION CARD	`_before(tape, j) · line 456`
DOES	Replays the tape up to slot j.
RETURNS	A pair (our position, Q = net lots we bought at their quotes).
WHY IT MATTERS	Each past event has to be read in the situation it happened in.
CALLED BY	_see, _quote, _respond, _move.

```
455     @staticmethod
456     def _before(tape, j):
457         """My position and my net lots bought at their quotes, before slot j."""
458         pos = 0
459         Q = 0
460         for k in range(j):
461             r, who, price, qty = tape[k]
462             if who == "me":
463                 pos -= qty
464             else:
465                 pos += qty
466                 Q += qty
467         return pos, Q
```

L460–466 Replay the tape up to slot j. When I was the maker, the taker's qty moves my position the other way (pos −= qty). When they were the maker, I was the taker (pos += qty), and Q counts the net lots I bought at their quotes.

Lines 469–477 · _ks: the sign swap-expecting bots use

FUNCTION CARD	`_ks(obs, r, pos) · line 470`
DOES	The direction a swap-expecting opponent assumes we will move V: the sign of our position while we can still swap.
RETURNS	+1, −1 or 0.
WHY IT MATTERS	Input to the kappa term of the opponent model.
CALLED BY	_see, _quote.

```
469     @staticmethod
470     def _ks(obs, r, pos):
471         """Sign used by swap-expecting bots: my position, if I may still swap."""
472         if r >= 5 or pos == 0:
473             return 0
474         for d in obs.discards:
475             if d[1] == "me" and d[0] <= r:
476                 return 0
477         return 1 if pos > 0 else -1
```

L472–477 +1 if I am long and can still swap, −1 if short, else 0. It is 0 in round 5 (all swaps happen before round 5's quotes) and after I have swapped. Opponents with kappa > 0 shift their value by kappa × this, expecting me to swap away a bad card.

Lines 479–495 · _lastq: my last quote, carried forward

FUNCTION CARD	`_lastq(obs, j, r) · line 480`
DOES	Our last quote before slot j, carried forward for new board cards and our swaps the way a blend quoter carries it.
RETURNS	A number, or None if we have not quoted.
WHY IT MATTERS	Blend quoters copy part of this into their own quote.
CALLED BY	_see, _respond, _move.

```
479     @staticmethod
480     def _lastq(obs, j, r):
481         """My last quote before tape slot j, carried to round r the way a
482         blend quoter carries it (board cards and my swaps since), or None."""
483         tape = obs.tape
484         k = min(j, len(tape)) - 1
485         while k >= 0 and tape[k][1] != "me":
486             k -= 1
487         if k < 0:
488             return None
489         rq, q = tape[k][0], tape[k][2]
490         for c in obs.board[rq - 1:r - 1]:
491             q += CARRY * (c - 7.0)
492         for d in obs.discards:
493             if d[1] == "me" and rq < d[0] <= r:
494                 q += 7.0 - d[2]
495         return q
```

L484–488 Find my newest quote before slot j.

L489–491 Each board card shown since adds CARRY × (card − 7).

L492–494 Each of my swaps since adds 7 − (the card I threw).

L495 What a blend quoter would now take from my old quote.

Lines 497–514 · _see, part 1: one observation

FUNCTION CARD	`_see(obs, r, j, price, qty, is_quote) · line 497`
DOES	Learns from one event for every hypothesis: from their quote with a Kalman update, or from their answer to our quote with a truncated-normal update; then reweights and prunes.
RETURNS	Nothing; updates self.H.
WHY IT MATTERS	The heart of the bot's learning. Everything it believes about the opponent comes from here.
CALLED BY	_sync.

```
497     def _see(self, obs, r, j, price, qty, is_quote):
498         tape = obs.tape
499         st = self._state(obs, r, self.obar)
500         pos, Q = self._before(tape, j)
501         ks = self._ks(obs, r, pos)
502         mq = [tape[k][2] for k in range(min(j, len(tape))) if tape[k][1] == "me"]
503         mqc = mq + [price] if not is_quote else mq
504         if not is_quote:
505             can_buy = pos > -10
506             can_sell = pos < 10
507             if qty > 0:
508                 kind = 1
509             elif qty < 0:
510                 kind = -1
511             else:
512                 kind = 0
513                 if not can_buy and not can_sell:
514                     return
```

_see is called once per event: their quote (is_quote = True, price = their quote) or their answer to my quote (is_quote = False, price = my quote, qty = their answer).

L499–501 The public state at that time, my position, Q and ks.

L502–503 My quotes so far; a CUR reader answering also reads the very price it is answering.

L504–514 For an answer: kind = +1 (bought), −1 (sold) or 0 (passed). If the position limit blocked both buying and selling, a pass says nothing, so return.

Lines 515–530 · _see, part 2: per-hypothesis set-up

```
515         cof = {}
516         lq = self._lastq(obs, j, r) if is_quote else None
517         for h in self.H:
518             reads, rule, lam = h[H_READS], h[H_RULE], h[H_LAM]
519             if is_quote and lam == 0:
520                 h[H_LW] += math.log(U_Q)
521                 continue
522             key = (reads, rule)
523             c = cof.get(key)
524             if c is None:
525                 src = mqc if (rule == CUR and not is_quote) else mq
526                 q, L = pick_q(st, rule, src)
527                 c = cof[key] = coefs(st, reads, rule, q, L)
528             A0, A1, B0, B1 = c
529             mo, mt = h[H_MO], h[H_MT]
530             extra = h[H_TAU] * Q + h[H_KAP] * ks
```

L515 A cache of line coefficients per (reads, rule): all hypotheses of a family share them.

L516 My carried last quote, for blend quoters.

L519–521 A lam = 0 hypothesis says "their quotes carry no information": the quote gets the flat density U_Q and the belief does not change.

L522–527 The coefficients for this family, from the cache or computed with pick_q and coefs.

L529–530 The belief means; extra = tau·Q + kappa·ks.

Lines 531–553 · _see, part 3: learning from their quote

```
            if is_quote:
                oe = lam * mo + (1 - lam) * self.obar
                if rule == BLD:
                    F = A0 + B0 * oe + extra
                    if lq is None:
                        pred, Ho, Ht = F, B0 * lam, 0.0
                    else:
                        F += h[H_SK] * pos
                        pred = (1 - mt) * F + mt * lq
                        Ho = (1 - mt) * B0 * lam
                        Ht = lq - F
                else:
                    pred = A0 + A1 * mt + (B0 + B1 * mt) * oe + extra
                    Ho = (B0 + B1 * mt) * lam
                    Ht = A1 + B1 * oe
                S = (h[H_P00] * Ho * Ho + 2 * h[H_P01] * Ho * Ht + h[H_P11] * Ht * Ht
                     + SIG_Q * SIG_Q)
                d = price - pred
                dens = math.exp(-d * d / (2 * S)) * ISQ2PI / math.sqrt(S)
                ev = (1 - EPS_Q) * dens + EPS_Q * U_Q
                rr = (1 - EPS_Q) * dens / ev
                ekf_mix(h, rr, Ho, Ht, d, S, 0.0)
                h[H_LW] += math.log(ev)
```

L532 oe = lam·O + (1 − lam)·obar: a lam-honest quoter shows only the fraction lam of its hand's distance from average.

L533–541 Blend quoter: its own value F (plus skew × my position), then quote = (1 − W)·F + W·(my carried last quote). Ho and Ht are the slopes of the predicted quote in O and in W.

L542–545 Other styles: the predicted quote from the straight line, and its slopes Ho and Ht.

L546–547 The predicted variance S = $H^T \cdot P \cdot H$ + SIG_Q².

L548 d = the surprise.

L549 dens = the normal density of the surprise.

L550 The likelihood with outliers: 90% normal, 10% flat.

L551 rr = the chance that the quote was explained.

L552 The Kalman step, blended by rr.

L553 Add the log-likelihood to the weight.

Lines 554–590 · _see, part 4: learning from their answer to my quote

```
            else:
                th = h[H_TH]
                if kind > 0:
                    lo, hi = price + th, INF
                elif kind < 0:
                    lo, hi = -INF, price - th
                else:
                    lo = price - th if can_sell else -INF
                    hi = price + th if can_buy else INF
                my = A0 + A1 * mt + (B0 + B1 * mt) * mo + extra
                Ho = B0 + B1 * mt
                Ht = A1 + B1 * mo
                S = (h[H_P00] * Ho * Ho + 2 * h[H_P01] * Ho * Ht + h[H_P11] * Ht * Ht
                     + SIG_R * SIG_R)
                sy = math.sqrt(S)
                if lo == -INF:
                    al, pa, Fa, ta = -INF, 0.0, 0.0, 0.0
                else:
                    al = (lo - my) / sy
                    pa, Fa = pdf(al), Phi(al)
                    ta = al * pa
                if hi == INF:
                    pb, Fb, tb = 0.0, 1.0, 0.0
                else:
                    be = (hi - my) / sy
                    pb, Fb = pdf(be), Phi(be)
                    tb = be * pb
                Z = Fb - Fa
                ev = (1 - EPS_R) * Z + EPS_R / 3.0
                h[H_LW] += math.log(ev)
```

```
584                 if Z < 1e-9:
585                     continue
586                 rr = (1 - EPS_R) * Z / ev
587                 m1 = (pa - pb) / Z
588                 vy = S * (1 + (ta - tb) / Z - m1 * m1)
589                 ekf_mix(h, rr, Ho, Ht, sy * m1, S, vy)
590         self._prune()
```

L555 Their threshold th.

L556–562 The interval their value must be in: bought → above price + th; sold → below price − th; passed → within ±th, except that a side blocked by the position limit is left open.

L563–568 The predicted value (mean), its slopes, the predicted variance S (plus SIG_R² for threshold noise) and its sd sy.

L569–580 The standardised ends of the interval, al and be, with their density and cumulative values (infinite ends handled).

L581 Z = the probability of this answer.

L582 The likelihood, allowing 6% random answers (EPS_R / 3 for each kind of answer).

L583 Update the weight.

L584–585 If the answer was impossible under this hypothesis, skip the belief update.

L586–588 rr; m1 = the mean of the truncated normal (in sd units); vy = its variance.

L589 The Kalman step: the observation "came out" sy·m1 above the prediction, with vy of uncertainty left.

L590 Prune after every event.

> **SAY IT LIKE THIS**
> "From a quote we learn a number; from an answer we learn an interval. Both go through the same Kalman step, and each hypothesis' weight goes up or down by how likely it found what happened."

Lines 592–599 · _see_swap: the opponent swapped

FUNCTION CARD	`_see_swap(obs, card) · line 592`
DOES	Updates every belief after the opponent swaps: O moves by (average new card − thrown card) and becomes more uncertain.
RETURNS	Nothing.
WHY IT MATTERS	A swap changes the opponent's hand; ignoring it would spoil every estimate after it.
CALLED BY	_sync.

```
592     def _see_swap(self, obs, card):
593         u = obs.unseen
594         U = sum(u)
595         m = sum((r + 1) * c for r, c in enumerate(u)) / U
596         v = sum(c * (r + 1 - m) ** 2 for r, c in enumerate(u)) / U
597         for h in self.H:
598             h[H_MO] += m - card
599             h[H_P00] += v
```

L593–596 The mean m and variance v of a random unseen card.

L597–599 Their new hand = old hand − thrown card + new card, so in every hypothesis the mean of O moves by m − card and its variance grows by v.

Lines 601–630 · _sync: process everything new, in order

FUNCTION CARD	_sync(obs, cur=None) · line 601
DOES	Processes everything new since the last call in game order (swaps before the quotes of the same round), and the opponent's current quote when we are answering it.
RETURNS	Nothing.
WHY IT MATTERS	Runs at the start of every decision so beliefs are always up to date; also builds the hypotheses on the first call.
CALLED BY	_quote, _respond, _move.

```
601     def _sync(self, obs, cur=None):
602         if self.H is None:
603             self._init(obs)
604         tape = obs.tape
605         disc = obs.discards
606         while True:
607             nd = disc[self.ndisc] if self.ndisc < len(disc) else None
608             nt = tape[self.done] if self.done < len(tape) else None
609             if nd is None and nt is None:
610                 break
611             if nd is not None and (nt is None or nd[0] <= nt[0]):
612                 if nd[1] == "opp":
613                     self._see_swap(obs, nd[2])
614                 self.ndisc += 1
615                 continue
616             j = self.done
617             self.done += 1
618             if j == self.extra:
619                 continue
620             r, who, price, qty = nt
621             if who == "opp":
622                 if 0.0 < price < MAXP:
623                     self._see(obs, r, j, price, 0, True)
624             else:
625                 self._see(obs, r, j, price, qty, False)
626         if cur is not None:
627             j = len(tape)
628             if 0.0 < cur < MAXP:
629                 self._see(obs, obs.round, j, cur, 0, True)
630             self.extra = j
```

L602–603 First call: build the hypotheses.

L606–615 Walk the discards and the tape together. A discard of round r happens before the quotes of round r, so it goes first. Only opponent discards change our beliefs.

L616–619 Skip a quote we already learned from early.

L620–625 Their quote (ignoring the extreme prices 0 and 195, which carry no hand information) → quote update. My quote → answer update.

L626–630 When answering (cur given), learn from their current quote now, and remember its slot so it is not used twice.

Lines 632–643 · _ev: E[V | O] = α + β·O

FUNCTION CARD	_ev(obs) · line 632
DOES	The expected value of the stock as a straight line in the opponent's hand sum.
RETURNS	A pair (alpha, beta) with E[V | O] = alpha + beta·O.
WHY IT MATTERS	Turns beliefs about O into the money value of our position and trades.
CALLED BY	_quote, _respond, _move.

```
632     def _ev(self, obs):
633         """E[V | O] = alpha + beta * O."""
634         u = obs.unseen
635         U = sum(u)
```

```
636         S = sum((r + 1) * c for r, c in enumerate(u))
637         k = 5 - len(obs.opp_known)
638         kn = sum(obs.opp_known)
639         hb = 5 - len(obs.board)
640         den = U - k
641         alpha = sum(obs.hand) + sum(obs.board) + hb * (S + kn) / den
642         beta = 1 - hb / den
643         return alpha, beta
```

L634–636 Unseen count and sum (their unknown cards are among them).

L637–638 k = their cards I have not peeked (always 5 for us); kn = the sum of the peeked ones.

L639 The hidden board cards.

L640–642 The hidden board comes from the unseen pool minus their k unknown cards, which gives α and β as in section 3.1.

Lines 645–655 · _opp_swap (not used any more)

FUNCTION CARD	_opp_swap(obs) · line 646
DOES	Estimates how the opponent's coming swap would move V. Not called any more.
RETURNS	A number.
WHY IT MATTERS	An idea from moscow; palermo stopped using it after live tests showed it cost points against top teams.
CALLED BY	Nothing.

```
645     @staticmethod
646     def _opp_swap(obs):
647         """The swap the opponent still has (live teams swap in round 4-5): when I am short it
648         is long and throws a low card, so V goes up by about m - (a low card); when I am long, down."""
649         if obs.round >= 5 or obs.position == 0 or any(w == "opp" for _, w, _ in obs.discards):
650             return 0.0
651         pool = sorted(r + 1 for r, c in enumerate(obs.unseen) for _ in range(c))
652         n = len(pool)
653         m = sum(pool) / n
654         card = pool[n // 6] if obs.position < 0 else pool[(5 * n) // 6]
655         return OPP_SWAP_K * (m - card) if abs(m - card) > 3 else 0.0
```

An idea from moscow (v05): price in the swap the opponent may still make. palermo (v10) stopped calling it, because live tests showed it cost points against the top teams. The function is still in the file but is never called.

Lines 657–664 · _mo: our best guess of O

FUNCTION CARD	_mo() · line 657
DOES	Our single best guess of O: the weighted average over all hypotheses.
RETURNS	A number.
WHY IT MATTERS	Used for the safe quote's direction and the swap's card estimate.
CALLED BY	_quote, _move.

```
657     def _mo(self):
658         tw = 0.0
659         tm = 0.0
660         for h in self.H:
661             w = math.exp(h[H_LW])
662             tw += w
663             tm += w * h[H_MO]
664         return tm / tw
```

The weighted average of O over all hypotheses.

Lines 666–678 · _p_reader

FUNCTION CARD	`_p_reader() · line 666`
DOES	The probability that the opponent is a confident, informative reader, and that it reads raw or current quotes.
RETURNS	A pair (p_reader, p_raw).
WHY IT MATTERS	The gate for paid manipulation, which is switched off (P_EXT and P_PROBE are 2.0).
CALLED BY	_quote.

```
666     def _p_reader(self):
667         """P(reads my quotes with trust >= 0.5), P(... and without a clamp)."""
668         tw = 0.0
669         trd = 0.0
670         trw = 0.0
671         for h in self.H:
672             w = math.exp(h[H_LW])
673             tw += w
674             if h[H_READS] and h[H_MT] >= 0.5 and h[H_LAM] >= LAM_PROBE:
675                 trd += w
676                 if h[H_RULE] in (RAW, CUR):
677                     trw += w
678         return trd / tw, trw / tw
```

The probability that the opponent is a confident, informative reader (trust ≥ 0.5, lam ≥ 0.4), and the probability of that with rule RAW or CUR. Only used for paid manipulation, which is switched off (P_EXT = P_PROBE = 2.0).

Lines 680–716 · _exact_reader: does the opponent fit the house reader formula?

FUNCTION CARD	`_exact_reader(obs) · line 680`
DOES	Tests whether the opponent's last 3 or more quotes fit the house reader formula exactly for one trust t.
RETURNS	t, or None.
WHY IT MATTERS	Allows free extreme quotes against an exact house reader in rounds 1–4.
CALLED BY	_quote.

```
680     def _exact_reader(self, obs):
681         """Trust t if their quotes since their last swap (3 or more) fit the house
682         reader formula exactly (raw reading of my last quote, one trust t, no
683         trade or swap terms), else None."""
684         last_swap = 0
685         for d in obs.discards:
686             if d[1] == "opp" and d[0] > last_swap:
687                 last_swap = d[0]
688         rows = []
689         lastq = None
690         for r, who, price, qty in obs.tape:
691             if who == "me":
692                 lastq = price
693                 continue
694             if r < last_swap or not 0.0 < price < MAXP:
695                 continue
696             rows.append((price,) + coefs(self._state(obs, r, self.obar), True, RAW, lastq, None))
697         if len(rows) < 3:
698             return None
699         best, bt = None, None
700         for i in range(231):
701             t = i * 0.005
702             num = 0.0
703             den = 0.0
704             for q, A0, A1, B0, B1 in rows:
705                 b = B0 + B1 * t
706                 num += (q - A0 - A1 * t) * b
707                 den += b * b
708             O = num / den
709             err = 0.0
```

```
            for q, A0, A1, B0, B1 in rows:
                e = abs(q - A0 - A1 * t - (B0 + B1 * t) * O)
                if e > err:
                    err = e
            if best is None or err < best:
                best, bt = err, t
        return bt if best < EX_TOL else None
```

L684–687 Their last swap round (quotes before it used another hand).

L688–696 rows: each of their quotes since then (not extreme), with the straight-line coefficients of a RAW reader of my last quote.

L697–698 At least 3 are needed.

L699–715 For every trust t from 0 to 1.15 in steps of 0.005: the best-fitting O (least squares), then the largest misfit. Keep the t with the smallest largest misfit.

L716 Return t if every quote fits within 0.02, else None. _quote adds extreme prices when t is about 0.8 (the house reader).

Your notes

8. The Bot: planning and decisions (lines 718–1078)

How the bot chooses a quote, an answer and a move.

Lines 718–728 · _slots: who quotes in each remaining slot

FUNCTION CARD	`_slots(first, start) · line 720`
DOES	Lists the remaining quote slots and who is the maker in each.
RETURNS	A list of (we are the maker?, round).
WHY IT MATTERS	The play-outs need the order of the remaining quotes.
CALLED BY	_plan_setup, _move.

```
718     # ── planning ──────────────────────────────────────────
719     @staticmethod
720     def _slots(first, start):
721         """Quote slots start..9 as (I am the maker, round). Each round has two
722         slots; I quote first in odd rounds iff `first`."""
723         out = []
724         for s in range(start, 10):
725             r = s // 2 + 1
726             me_first = (r % 2 == 1) == first
727             out.append(((s % 2 == 0) == me_first, r))
728         return out
```

L724–727 Slot s (0 to 9) is in round s // 2 + 1. The first slot of a round belongs to the player who starts that round, and I start the odd rounds if first is True.

Example with first = True: (me, 1) (opp, 1) (opp, 2) (me, 2) (me, 3) (opp, 3) (opp, 4) (me, 4) (me, 5) (opp, 5).

Lines 730–746 · _bel: their value under one hypothesis

FUNCTION CARD	`_bel(st, h, O, q, Q, ks) · line 730`
DOES	The opponent's fair value under one hypothesis with point values, reading our quote q.
RETURNS	A pair (fair value, slope in O).
WHY IT MATTERS	The quick form of the opponent model used inside play-outs and the answer model.
CALLED BY	_roll, resp_dist_raw inside _quote.

```
730     def _bel(self, st, h, O, q, Q, ks):
731         """Opponent fair under hypothesis h (point estimates) reading my quote q."""
732         L = None
733         if q is not None and h[H_READS]:
734             rule = h[H_RULE]
735             if rule == SANE:
736                 if not 12 <= implied(st, q) <= 58:
737                     q = None
738             elif rule == CLAMP:
739                 imp = implied(st, q)
740                 if imp > 65:
741                     L = 65.0
742                 elif imp < 5:
743                     L = 5.0
744         A0, A1, B0, B1 = coefs(st, h[H_READS], h[H_RULE], q, L)
745         t = h[H_MT]
746         return A0 + A1 * t + (B0 + B1 * t) * O + h[H_TAU] * Q + h[H_KAP] * ks, B0 + B1 * t
```

L733–743 The reading rules: SANE ignores implausible quotes; CLAMP caps the implied hand.

L744–746 fair = A0 + A1·t + (B0 + B1·t)·O + tau·Q + kappa·ks, returned with its slope in O.

Lines 748–760 · _roll: set-up

FUNCTION CARD	`_roll(st, h, V, pos, Q, qeff, slots, swapped, lq) · line 748`
DOES	Plays out the rest of the deal quickly under one hypothesis, with simple rules for both sides.
RETURNS	A pair (our profit relative to V, our final position).
WHY IT MATTERS	Values what an action does to the future (our position, what readers will believe next), not just the trade now. Also gives the expected final position for the swap.
CALLED BY	_quote, _respond, _move.

```
748     def _roll(self, st, h, V, pos, Q, qeff, slots, swapped, lq=None):
749         """Deterministic play-out of `slots` under hypothesis h, with simple
750         rules for me (trade against their belief, extreme quote at +-10).
751         Returns (profit relative to value V, final position)."""
752         O = h[H_MO]
753         lam = h[H_LAM]
754         reads = h[H_READS]
755         rule = h[H_RULE]
756         cur_rule = reads and rule == CUR
757         bld = rule == BLD
758         W = h[H_MT]
759         sk = h[H_SK]
760         pnl = 0.0
```

A fast, deterministic play-out of the remaining slots under one hypothesis, used to value what an action does to the rest of the deal. V is the value of the stock under this hypothesis; all profits are measured against it.

Lines 761–789 · _roll: my quotes

```
761         for me, r in slots:
762             ks = 0 if (swapped or r >= 5 or pos == 0) else (1 if pos > 0 else -1)
763             f, b = self._bel(st, h, O, qeff, Q, ks)
764             if me:
765                 if pos >= 10:
766                     p = MAXP
767                 elif pos <= -10:
768                     p = 0.0
769                 elif f > V + 1:
770                     p = f - 3.05
771                 elif f < V - 1:
772                     p = f + 3.05
773                 else:
774                     p = f
775                 if cur_rule:
776                     f, b = self._bel(st, h, O, p, Q, ks)
777                 e = f - p
778                 th = h[H_TH]
779                 if e > th:
780                     n = min(2, 10 + pos)
781                     pos -= n
782                     pnl += n * (p + 2 - V)
783                 elif e < -th:
784                     n = min(2, 10 - pos)
785                     pos += n
786                     pnl += n * (V - p + 2)
787                 if reads and (rule != SANE or 12 <= implied(st, p) <= 58):
788                     qeff = p
789                 lq = p
```

L762 ks for this slot.

L763 Their value f (and its slope b).

L765–768 At +10 lots I quote 195, at −10 I quote 0: only the trade that helps me is possible, and readers are pulled my way.

L769–774 If they value the stock more than 1 above V, quote 3.05 below their value so they buy (I sell above V); if more than 1 below, quote 3.05 above so they sell; otherwise quote their value.

L775–776 A CUR reader re-reads this price.

L777–786 If their edge beats their threshold, they trade 2 lots (capped by the limit); my profit is counted at V.

L787–789 Readers now read this quote (SANE ones only if it is plausible).

Lines 790–804 · _roll: their quotes

```
            elif lam > 0:
                q = f - b * (1 - lam) * (O - self.obar)
                if bld and lq is not None:
                    q = (1 - W) * (q + sk * pos) + W * lq
                if V - q - 2 > 0:
                    n = min(2, 10 - pos)
                    pos += n
                    Q += n
                    pnl += n * (V - q - 2)
                elif q - 2 - V > 0:
                    n = min(2, 10 + pos)
                    pos -= n
                    Q -= n
                    pnl += n * (q - 2 - V)
        return pnl, pos
```

L790 lam = 0 quoters are not simulated (their quotes carry nothing).

L791 Their quote = their value, with only the fraction lam of their hand's distance from average showing.

L792–793 Blend quoters mix in my last quote (and their skew).

L794–803 I buy 2 if V − price − 2 > 0 and sell 2 if price − 2 − V > 0; Q keeps track of the lots I bought at their quotes.

L804 Return (profit relative to V, final position).

Lines 806–818 · _plan_setup

FUNCTION CARD	**_plan_setup(obs, k) · line 806**
DOES	Gathers what every decision needs: the public state, the 8 heaviest hypotheses and their weights, which quote each reader rule reads, whether we have swapped, and the remaining slots.
RETURNS	A tuple (st, hs, ws, qe, slots, swapped).
WHY IT MATTERS	One shared setup, so quoting and answering use the same view.
CALLED BY	_quote, _respond.

```
    def _plan_setup(self, obs, k):
        st = self._state(obs, obs.round, self.obar)
        hs = heapq.nlargest(NPLAN, self.H, key=lambda h: h[H_LW])
        ws = [math.exp(h[H_LW]) for h in hs]
        tw = sum(ws)
        ws = [w / tw for w in ws]
        mq = [e[2] for e in obs.tape if e[1] == "me"]
        qe = {}
        for rule in (RAW, CLAMP, SANE, CUR, BLD):
            q = pick_q(st, rule, mq)[0]
            qe[rule] = q
        swapped = any(d[1] == "me" for d in obs.discards)
        return st, hs, ws, qe, self._slots(self.first, k + 1), swapped
```

L807 The public state now.

L808–811 The 8 heaviest hypotheses and their normalised weights.

L812–816 For each reading rule, which of my quotes a reader would use now.

L817 Have I swapped?

L818 The remaining slots after the current one.

Lines 820–838 · _public (not used) and _plain_fair (the fallback)

FUNCTION CARD	`_public(obs) · _plain_fair(obs) · line 822`
DOES	_public: value from public cards only (not used any more). _plain_fair: a simple card-counting value of the stock.
RETURNS	A number each.
WHY IT MATTERS	_plain_fair is the safety net: if the main code ever fails, the bot still quotes and answers sensibly.
CALLED BY	_plain_fair: quote and respond, on errors.

```
    # ── the three methods the engine calls ──────────────────────
    @staticmethod
    def _public(obs):
        """What an average hand on both sides implies, from public cards only."""
        u = [4] * 13
        for k in obs.board:
            u[k - 1] -= 1
        for _, _, k in obs.discards:
            u[k - 1] -= 1
        mp = sum((r + 1) * c for r, c in enumerate(u)) / sum(u)
        return sum(obs.board) + (15 - len(obs.board)) * mp

    @staticmethod
    def _plain_fair(obs):
        """Card-counting value, used only if something goes wrong."""
        u = obs.unseen
        m = sum((r + 1) * c for r, c in enumerate(u)) / max(1, sum(u))
        hidden = 10 - len(obs.opp_known) - len(obs.board)
        return sum(obs.hand) + sum(obs.board) + sum(obs.opp_known) + hidden * m
```

L822–830 _public: the value implied by the public cards alone (moscow's "first quote = public estimate"; no longer called since palermo).

L833–838 _plain_fair: card counting: my hand + board + peeked cards + every hidden card at the unseen average. Used only if the main code raises an error.

Lines 840–871 · The three methods the engine calls

FUNCTION CARD	`quote(obs) · respond(obs, price) · move(obs) · line 840`
DOES	The three methods the engine calls. Each asks the real decision function, falls back to a simple rule if it raises an error, and (for quote and respond) lets the house-bot attack take over when it is active.
RETURNS	quote: our price. respond: lots to buy (+) or sell (−), at most 2. move: None or ("SWAP", card index).
WHY IT MATTERS	They are the bot's only contact with the game, so they must never crash; a bug costs a little edge, never a deal.
CALLED BY	The game engine.

```
    def quote(self, obs):
        try:
            q = self._quote(obs)
        except Exception:
            q = self._plain_fair(obs)
        try:
            self.rd.sync(obs)
            if self.rd.active(obs):
                return self.rd.quote(obs)
        except Exception:
            self.rd.off = True
        return q

    def respond(self, obs, price):
        try:
            a = self._respond(obs, price)
```

```
856         except Exception:
857             e = self._plain_fair(obs) - price
858             a = 2 if e > 2 else (-2 if e < -2 else 0)
859         try:
860             self.rd.sync(obs, price)
861             if self.rd.active(obs):
862                 return self.rd.respond(obs, price)
863         except Exception:
864             self.rd.off = True
865         return a
866 
867     def move(self, obs):
868         try:
869             return self._move(obs)
870         except Exception:
871             return None
```

L840–844 quote: try the real thing; on any error use the card-counting value.

L845–851 Then the house-reader layer may replace it (only when active). If that layer errors, it is switched off.

L853–865 respond: the same pattern. The fallback trades 2 lots if the card-counting edge is over 2.

L867–871 move: on an error, do nothing.

> **SAY IT LIKE THIS**
> "Every engine call is wrapped: if anything goes wrong we fall back to a simple card-counting value, so a bug costs a little edge, never a crash."

Lines 873–890 · _quote, part 1: set-up

FUNCTION CARD	`_quote(obs) · line 873`
DOES	Chooses our price as the maker: updates beliefs, models how the opponent will answer each price, finds the best price for this trade alone (stage 1), scores a short list together with the rest of the deal through the utility (stage 2), then moves the result 30% toward the safe quote.
RETURNS	The price, between 0 and 195.
WHY IT MATTERS	Our main decision as the maker, where the edge is earned and our hand is partly hidden.
CALLED BY	quote.

```
873     def _quote(self, obs):
874         self._sync(obs)
875         tape = obs.tape
876         j = len(tape)
877         if self.first is None:
878             self.first = (j == 0)
879         pos = obs.position
880         _, Q = self._before(tape, j)
881         alpha, beta = self._ev(obs)
882         st, hs, ws, qe, slots, swapped = self._plan_setup(obs, j)
883         ks = self._ks(obs, obs.round, pos)
884         cur = obs.cash - obs.spent
885         nb = min(2, 10 + pos)
886         ns = min(2, 10 - pos)
887         comps = []
888         for w, h in zip(ws, hs):
889             V = alpha + beta * h[H_MO]
890             comps.append((w, h, V))
```

L874 Learn from everything new.

L876–878 j = the tape length. At my first call: if the tape is empty, I am the first maker (seat 0, odd rounds).

L879–886 Position, Q, α and β, the planning set-up, ks, the money so far (cur), and how many lots they can buy (nb) or sell (ns) at my quote.

L887–890 For each top hypothesis: its weight, the hypothesis, and V = α + β·O.

Lines 892–919 · _quote, part 2: their answer model

FUNCTION CARD	resp_dist(h, p) · resp_dist_raw(h, p) (inside _quote) · line 894
DOES	The opponent's fair value when answering price p under hypothesis h: its mean, its spread, and how it moves with O. resp_dist caches it per hypothesis because only CUR readers depend on p.
RETURNS	A tuple (mean f, sd, cov(O, fair) / sd).
WHY IT MATTERS	The input to the expected-profit formula; the cache was part of marseille's speed-up.
CALLED BY	imm and stage 1 inside _quote.

```
892         rd_cache = {}
893
894         def resp_dist(h, p):
895             """Mean, sd and cov(O, .)/sd of their fair when answering price p (cached when
896             their answer does not read p, which is every hypothesis but the CUR readers)."""
897             if not (h[H_READS] and h[H_RULE] == CUR):
898                 got = rd_cache.get(id(h))
899                 if got is None:
900                     got = rd_cache[id(h)] = resp_dist_raw(h, p)
901                 return got
902             return resp_dist_raw(h, p)
903
904         def resp_dist_raw(h, p):
905             q = p if (h[H_READS] and h[H_RULE] == CUR) else qe[h[H_RULE]]
906             f, _ = self._bel(st, h, h[H_MO], q, Q, ks)
907             L = None
908             if q is not None and h[H_READS] and h[H_RULE] == CLAMP:
909                 imp = implied(st, q)
910                 L = 65.0 if imp > 65 else (5.0 if imp < 5 else None)
911             if q is not None and h[H_READS] and h[H_RULE] == SANE and not 12 <= implied(st, q) <= 58:
912                 q = None
913             A0, A1, B0, B1 = coefs(st, h[H_READS], h[H_RULE], q, L)
914             Ho = B0 + B1 * h[H_MT]
915             Ht = A1 + B1 * h[H_MO]
916             cO = h[H_P00] * Ho + h[H_P01] * Ht
917             S = cO * Ho + (h[H_P01] * Ho + h[H_P11] * Ht) * Ht + SIG_R * SIG_R
918             sf = math.sqrt(S)
919             return f, sf, cO / sf
```

L894–902 resp_dist with a per-quote cache: for every hypothesis except CUR readers, the answer model does not depend on my price, so it is computed once (part of marseille's speed-up).

L905 Which quote of mine they read: the price itself for CUR, otherwise their current reading.

L906 Their value f.

L907–913 The reading rules and coefficients.

L914–919 The slopes; cO = cov(O, their value); S = var(their value) + SIG_R². Returns the mean, the sd and cO / sd.

Lines 921–927 · _quote, part 3: the immediate expected profit

FUNCTION CARD	imm(h, V, p) (inside _quote) · line 921
DOES	The expected profit of quoting p right now under hypothesis h, including adverse selection: the opponent buys exactly when its hand, and so V, is high.
RETURNS	A pair (expected profit, their fair value).
WHY IT MATTERS	The core of pricing: what a price earns on average against an opponent who trades when it suits them.
CALLED BY	Stages 1 and 2 of _quote.

```
921         def imm(h, V, p):
922             f, sf, c = resp_dist(h, p)
923             th = h[H_TH]
924             zb = (f - p - th) / sf
```

```
925             zs = (p - th - f) / sf
926             return (nb * ((p + 2 - V) * Phi(zb) - beta * c * pdf(zb))
927                     + ns * ((V - p + 2) * Phi(zs) - beta * c * pdf(zs))), f
```

L923–925 The z-scores of "they buy" (value − p > th) and "they sell" (p − value > th).

L926 nb lots sold at p + 2 with probability Φ(zb), minus the adverse-selection term β·c·φ(zb): they buy exactly when O, and so V, is high.

L927 The same for selling to me. Returns (expected profit, their value).

> **SAY IT LIKE THIS**
> "The immediate profit of a quote includes adverse selection: the opponent buys exactly when its hand is high, so the formula subtracts β times the covariance times the normal density."

Lines 929–942 · _quote, part 4: stage 1, the best price for this trade alone

```
929         # the best price for this trade alone (risk-neutral)
930         f0 = [resp_dist(h, comps[0][2])[0] for _, h, _ in comps[:2]]
931         cands = set()
932         for f in f0:
933             for d in DELTAS:
934                 cands.add(f + d)
935                 cands.add(f - d)
936         best, bp = 0.0, f0[0]
937         for p in cands:
938             v = 0.0
939             for w, h, V in comps:
940                 v += w * imm(h, V, p)[0]
941             if v > best:
942                 best, bp = v, p
```

L930 Their value under the top two hypotheses.

L931–935 Candidates: those values ± 2, 2.5, 3, 3.5, 4.5 and 6.

L936–942 Keep the candidate with the highest weighted immediate profit (default: their value, which means no trade).

Lines 943–960 · _quote, part 5: stage 2 candidates

```
943         # then weigh what each candidate does to the rest of the deal
944         cands = [bp, f0[0], f0[0] + 3.05, f0[0] - 3.05]
945         if len(f0) > 1:
946             cands += [f0[1] + 6.05, f0[1] - 6.05]
947         imp0 = st[0] + (5 + st[1]) * st[6]
948         if pos >= 10:
949             cands += [MAXP, imp0 + 65.0, imp0 + 58.0]
950         elif pos <= -10:
951             cands += [0.0, imp0 + 5.0, imp0 + 12.0]
952         elif obs.round <= 4 and slots and 0.79 <= (self._exact_reader(obs) or 0.0) <= 0.81:
953             cands += [MAXP, 0.0]
954         elif slots and not slots[0][0]:
955             # paid manipulation only when their next quote shows whether it worked
956             p_rd, p_raw = self._p_reader()
957             if p_raw >= P_EXT:
958                 cands += [MAXP, 0.0]
959             if p_rd >= P_PROBE:
960                 cands += [imp0 + 63.0, imp0 + 7.0, imp0 + 55.0, imp0 + 15.0]
```

L944–946 The stage-1 price, their value, ±3.05 around it, and ±6.05 around the second hypothesis' value.

L947 imp0 = the price that would imply a hand of 0 for me, so imp0 + x implies a hand of x.

L948–951 At the limit, extreme prices are free: at +10 the prices 195, "hand 65" and "hand 58"; at −10 the prices 0, "hand 5" and "hand 12".

L952–953 Against an exact house reader (trust about 0.8), extremes in rounds 1 to 4.

L954–960 Paid manipulation: switched off, because P_EXT and P_PROBE (2.0) can never be reached.

Lines 961–985 · _quote, part 6: score each candidate with the rest of the deal, then nudge

```
961         bestv, bestp = None, bp
962         for p in cands:
963             p = min(MAXP, max(0.0, p))
964             v = 0.0
965             for w, h, V in comps:
966                 x, f = imm(h, V, p)
967                 e = f - p
968                 ps = pos
969                 if e > h[H_TH]:
970                     ps -= nb
971                 elif e < -h[H_TH]:
972                     ps += ns
973                 qeff = p if h[H_READS] else None
974                 if h[H_READS] and h[H_RULE] == SANE and not 12 <= implied(st, p) <= 58:
975                     qeff = qe[SANE]
976                 x += self._roll(st, h, V, ps, Q, qeff, slots, swapped, p)[0]
977                 v += w * util(cur + pos * V + x)
978             if bestv is None or v > bestv:
979                 bestv, bestp = v, p
980         bestp = min(MAXP, max(0.0, bestp))
981         if -10 < pos < 10:
982             safe = self._safe_quote(obs, alpha + beta * self._mo())
983             if safe is not None:
984                 return (1 - SAFE_W) * bestp + SAFE_W * safe
985         return bestp
```

L962–963 Clamp to 0..195.

L965–972 For each hypothesis: the immediate profit, and the position after their likely answer.

L973–975 Readers will read this new price (SANE ones only if it is plausible).

L976 Add the play-out of the rest of the deal.

L977 The utility of the whole deal: money so far + position × V + future profit.

L978–979 Keep the best.

L980 Clamp.

L981–984 Not at the limit: final quote = 70% planner + 30% safe quote.

L985 At the limit: the planner's (extreme) price as it is.

WHY / EVIDENCE

Top teams quote close to the opponent's last quote and so hide their information. Moving 15% of the way to that "safe quote" (stockholm, v09) gained +0.46 ± 0.23 live; 30% (test u01) another +0.13; using it against every opponent (bogota, v11) another +0.06. Quoting only the safe quote (test j01) lost −0.85: we keep our edge and only borrow part of the hiding.

Lines 987–1003 · _safe_quote

FUNCTION CARD	`_safe_quote(obs, v) · line 988`
DOES	The opponent's last quote, carried for new board cards and swaps, moved 1.3 toward our value v.
RETURNS	A price, or None if the opponent has not quoted yet.
WHY IT MATTERS	The target of the 30% nudge: a price inside the opponent's no-trade band that reveals little about our hand.
CALLED BY	_quote.

```
987     @staticmethod
988     def _safe_quote(obs, v):
989         """Their last quote, carried for board cards and swaps since, plus SAFE_D towards my value v."""
990         last = None
991         for r, who, price, qty in obs.tape:
992             if who == "opp" and 0.0 < price < MAXP:
993                 last = (r, price)
994         if last is None:
```

```
            return None
        rq, q = last
        for c in obs.board[rq - 1:obs.round - 1]:
            q += c - 7.0
        for r, who, out in obs.discards:
            if r > rq:
                q += 7.0 - out
        q += SAFE_D if v > q else -SAFE_D
        return min(MAXP, max(0.0, q))
```

L990–995 Their last real quote (not 0 or 195). If there is none, there is no nudge.

L996–998 Carry it for the board cards shown since (each moves it by card − 7).

L999–1001 And for the swaps since (7 − the thrown card).

L1002 Move 1.3 towards my value: inside their ±2 band, so they should not trade on it.

L1003 Clamp.

Lines 1005–1035 · _respond: buy, pass or sell?

FUNCTION CARD	`_respond(obs, price) · line 1005`
DOES	Compares pass, buy and sell by the profit now plus a play-out of the rest of the deal under the top hypotheses, through the utility.
RETURNS	Lots: 0, +nbuy or −nsell (each at most 2, limited by the position cap).
WHY IT MATTERS	Our main decision as the taker.
CALLED BY	respond.

```
    def _respond(self, obs, price):
        self._sync(obs, price)
        tape = obs.tape
        j = len(tape)
        if self.first is None:
            self.first = (j != 0)
        pos = obs.position
        _, Q = self._before(tape, j)
        alpha, beta = self._ev(obs)
        st, hs, ws, qe, slots, swapped = self._plan_setup(obs, j)
        cur = obs.cash - obs.spent
        lq = self._lastq(obs, j, obs.round)
        nbuy = min(2, 10 - pos)
        nsell = min(2, 10 + pos)
        best, bq = None, 0
        for qty in (0, nbuy, -nsell):
            v = 0.0
            for w, h in zip(ws, hs):
                V = alpha + beta * h[H_MO]
                if qty > 0:
                    x = qty * (V - price - 2)
                elif qty < 0:
                    x = -qty * (price - 2 - V)
                else:
                    x = 0.0
                qeff = qe[h[H_RULE]] if h[H_READS] else None
                x += self._roll(st, h, V, pos + qty, Q + qty, qeff, slots, swapped, lq)[0]
                v += w * util(cur + pos * V + x)
            if best is None or v > best + 1e-9:
                best, bq = v, qty
        return bq
```

L1006 Learn from everything, including their current quote.

L1009–1010 If my first call is an answer at slot 0, they quoted first, so I am not first in odd rounds.

L1011–1018 Set-up; nbuy and nsell respect the limit.

L1020 Three options: pass, buy, sell.

L1023–1029 The profit now at this hypothesis' V (buy at price + 2, sell at price − 2).

L1030–1031 Plus the play-out of the rest of the deal (tau readers see the lots I traded through Q + qty).

L1032 The utility of the whole deal.

L1033–1034 Keep the best; ties go to the earlier option (pass first).

> **SAY IT LIKE THIS**
> "To answer a quote we compare buy, pass and sell with one yardstick: the profit now plus a play-out of the rest of the deal under the eight likeliest opponent models, through the utility."

Lines 1037–1078 · _move: never peek, swap only in round 5

FUNCTION CARD	`_move(obs) · line 1037`
DOES	Never peeks. In round 5, if we have not swapped, estimates our position as 0.7 × now + 0.3 × expected end, then swaps our lowest card if we will be long (highest if short) whenever the expected gain is positive.
RETURNS	None, or ("SWAP", index of the card to throw).
WHY IT MATTERS	The late swap was one of our biggest live gains: +0.86 for round 5 only, +0.10 for the zero bar, +0.40 for the position mix.
CALLED BY	move.

```
1037      def _move(self, obs):
1038          self._sync(obs)
1039          if obs.round < 5 or any(w == "me" for _, w, _ in obs.discards):
1040              return None
1041          pos = obs.position
1042          r = obs.round
1043          # expected final position, from play-outs of the rest of the deal
1044          st = self._state(obs, r, self.obar)
1045          hs = heapq.nlargest(NPLAN, self.H, key=lambda h: h[H_LW])
1046          ws = [math.exp(h[H_LW]) for h in hs]
1047          tw = sum(ws)
1048          alpha, beta = self._ev(obs)
1049          _, Q = self._before(obs.tape, len(obs.tape))
1050          mq = [e[2] for e in obs.tape if e[1] == "me"]
1051          qe = {rule: pick_q(st, rule, mq)[0] for rule in (RAW, CLAMP, SANE, CUR, BLD)}
1052          seats = (self.first,) if self.first is not None else (True, False)
1053          fpos = 0.0
1054          lq = self._lastq(obs, len(obs.tape), r)
1055          for first in seats:
1056              slots = self._slots(first, 2 * (r - 1))
1057              for w, h in zip(ws, hs):
1058                  V = alpha + beta * h[H_MO]
1059                  qeff = qe[h[H_RULE]] if h[H_READS] else None
1060                  fpos += w * self._roll(st, h, V, pos, Q, qeff, slots, False, lq)[1]
1061          fpos /= tw * len(seats)
1062          u = obs.unseen
1063          U = sum(u)
1064          S = sum((rk + 1) * c for rk, c in enumerate(u))
1065          k = 5 - len(obs.opp_known)
1066          eo = self._mo() - sum(obs.opp_known)
1067          mstack = (S - eo) / (U - k)
1068          hand = obs.hand
1069          fpos = 0.3 * fpos + 0.7 * pos
1070          if fpos > 0:
1071              i = min(range(5), key=lambda x: hand[x])
1072              gain = fpos * (mstack - hand[i])
1073          else:
1074              i = max(range(5), key=lambda x: hand[x])
1075              gain = fpos * (mstack - hand[i])
1076          if gain > SWAP_TH[r - 1]:
1077              return ("SWAP", i)
1078          return None
```

L1038 Keep the beliefs current.

L1039–1040 Do nothing before round 5, or if I have already swapped.

L1044–1061 The expected final position fpos: play out round 5's two quotes under the top hypotheses (both seatings if unknown).

L1062–1067 mstack = the expected value of a fresh card: the unseen total minus their expected unknown cards, divided by the remaining count.

L1068 My hand.

L1069 The swap position: 0.3 × the position I expect to end with (fpos) + 0.7 × the position I hold now (pos). The play-out tends to predict more round-5 trading than really happens, so leaning on the real position is steadier (pamplona used 0.5 / 0.5).

L1070–1075 Long → throw my lowest card; short → my highest. gain = (that mixed position) × (mstack − card).

L1076–1077 Swap if the expected gain is positive (SWAP_TH for round 5 is 0; it was 4 in marseille).

WHY / EVIDENCE

Swapping only in round 5 (rio, v03) gained +0.86 ± 0.39 live on real teams, and local tests of swapping from round 4 lost about 0.3. Lowering the round-5 bar from 4 to 0 (manila, v13) gained another +0.10 ± 0.05 live, and deciding on the average of the current and the expected final position (pamplona, v14) another +0.40 ± 0.13, better against 23 of 24 opponents; matias (v15) leans 70/30 towards the current position and plans with 8 hypotheses (better than pamplona against 22 of 29 opponents). By round 5 we know which way we are positioned, and the thrown card (which is public) leaves the opponent only two quotes to react.

WHY / EVIDENCE

We never peek: a peek costs 3 points for one random card (a fifth of the opponent's hand), while the opponent's quotes and answers already tell us most of O.

Your notes

9. Every parameter in one table

Name	Value	Meaning	How it was chosen
MAXP	195	highest price and highest V	13 × 15
P_NAIVE	0.3	prior: does not read my quotes	0.45 → 0.3 in oslo (+0.12 / +0.20 locally); 0.6 lost −0.91 live (aa01); 0.15 lost locally
P_BLEND	0.12	prior: blend quoter	0.25 not significant locally
BLEND_LAM	1 / 0.7 / 0.4 / 0.2	blend quoters: how much hand shows	from replays
BLEND_SK	0 / 0.9	blend quoters: skew per lot of my position	from replays
W_MEAN, W_SD	0.45, 0.25	blend weight on my last quote	0.7 (with P_BLEND 0.3) tested, worse
BLEND_TAU	0 / 1.5 / 3	blend quoters: move per lot I buy	
CARRY	0.82	how a shown card moves a carried quote	≈ 1 − 9/46 (round 2) to 1 − 6/43 (round 5)
RULE_PRIOR	.55 / .20 / .08 / .17	reader rules RAW / CLAMP / SANE / CUR	
LAM_PRIOR	1 / .7 / .4 / 0	honesty of their quotes (45 / 12 / 8 / 35%)	more weight on lam 0, or "liars", tested worse
TH_PRIOR	2 (75%), 7 (25%)	their take threshold	helsinki +0.86 locally; removing it −0.45 live; 60/40 −0.53 live
TAU_PRIOR	0 / 1.5 / 3	move per lot I buy (honest quoters)	
KAP_PRIOR	0 (80%), 3.9 (20%)	expects me to swap	
T_MEAN, T_SD	0.6, 0.3	reader trust prior	
SIG_Q	0.5	quote noise	1.0 and 1.5 tested: not clearly better
EPS_Q, U_Q	0.1, 0.02	unexplained quotes	
SIG_R	1.0	threshold noise	
EPS_R	0.06	random answers	
PRUNE	6	drop hypotheses this far behind	12 → 6 in marseille (speed, same play)
SWAP_TH	15 / 12 / 9 / 6 / 0	swap bar by round	only round 5 is used; 4 → 0 in manila (live +0.10 ± 0.05)
DELTAS	2 … 6	stage-1 price offsets	
NPLAN	8	hypotheses used to plan	6 → 8 in matias (all six local runs positive)
P_EXT, P_PROBE	2.0	paid manipulation off	stockholm: no probes
EX_TOL	0.02	exact-reader fit	
LAM_PROBE	0.4	probes only against informative readers	(probes off)
RISK	60	utility curvature	30 and 120 tested
SAFE_D	1.3	safe-quote step	2.0 tested live (bb01): a tie
SAFE_W	0.3	nudge share	0.15 then 0.3 gained live; 0.4 tied live (mm04: −0.00 ± 0.12)
OPP_SWAP_K	0.6	not used	moscow idea, removed in palermo
DECK_SUM	364	sum of the deck	4 × (1 + … + 13)
RD_T	0.8	house bots' trust in my quote	house formula
RD_TOL	0.0051	house quote rounding	half a cent
RD_NQ	2	quotes needed to identify a house bot	
STEER_IN, HARVEST_OUT	1.98, 3.02	inside / outside the house bots' band	

Name	Value	Meaning	How it was chosen
RESP_MARGIN	0.2	extra edge when answering a house bot	
DEPTH	4	exact search depth	3 in round 2 for speed

Your notes

10. How we got here

Version history

Every version changed one idea. "Live" is the paired result on the arena against real teams: the newer upload minus the older one, opponent by opponent on the same opponent uploads (± is two standard errors).

v	Name	What changed	Live result
00	tokyo	starting point: Bayesian hypotheses + planner	the best paired live record at the start
01	berlin	blend quoters with lam 0.2; exact house readers crushed	+0.07 and +0.11 vs tokyo (noise)
02	nairobi	exact house-reader attack (_Reader)	+7.4 vs house reader, +10.0 vs sharp; same play vs real teams
03	rio	swap only in round 5	+0.86 ± 0.39 vs nairobi
04	denver	first quote = public estimate	inside moscow
05	moscow	price in the opponent's coming swap	+0.04 ± 0.19 (neutral)
06	helsinki	take thresholds 2 / 7, P_NAIVE 0.45	−0.03 ± 0.34 (neutral live, +0.86 locally)
07	oslo	P_NAIVE 0.3	inside lisbon
08	lisbon	defence against double peekers	−0.02 ± 0.26; the defence was dropped
09	stockholm	15% nudge, no paid probes	+0.46 ± 0.23 vs lisbon
10	palermo	30% nudge; moscow's two ideas undone	+0.43 ± 0.38 vs the 30% test; Imposters +1.91
11	bogota	the nudge against every opponent	+0.06 ± 0.08 vs palermo
12	marseille	same play, 30–40% faster	+0.02 ± 0.07 vs bogota; 47 of 50 fights won
13	manila	swap in round 5 whenever the expected gain is positive	+0.10 ± 0.05 vs marseille; 45 of 49 fights won
14	pamplona	round-5 swap decided on the average of current and expected final position	+0.40 ± 0.13 vs manila; better on 23 of 24
15	matias	plan with 8 hypotheses; swap position 0.7 current + 0.3 expected	better than pamplona on 22 of 29 opponents, median +0.18

Ideas tested live and dropped

Test	Result
j01: quote only the safe quote	−0.85 ± 0.82: gives away our edge
aa01: P_NAIVE 0.6	−0.91 ± 0.64
z02: no take thresholds	−0.45 ± 0.61
bb01: safe-quote step 2.0	−0.07 ± 0.10 (tie)
ll01: opponent-swap pricing back	−0.02 ± 0.20 (tie)
mm03: threshold prior 60/40	−0.53 ± 0.32
mm04: nudge 40%	−0.00 ± 0.12 (tie: 30% stays)
rr02: play-out weighted 0.75 when quoting / answering	−0.36 ± 0.33 (14 opponents), −0.11 ± 0.27 (26): rejected, although it was +0.28 locally

How we tested

- **Locally**: against imitation bots of each live team, fitted to that team's public replays and calibrated so that our local scores against them match our live scores. One change at a time; a change was kept only if it was more than two standard errors better on two different sets of deals.

- **Live**: the arena plays every upload on the same deals against the same opponent uploads. Comparing two of our uploads opponent by opponent (a "paired" comparison) removes most of the luck. We kept a change only if it did not lose live, and the biggest steps (the round-5 swap and the nudge) were found and confirmed this way.
- We learned about other teams only from the public arena pages (standings, fight lists, replays). No other team's code was used.

Your notes

11. Questions the organisers may ask

Q. Explain your bot in one minute.

The stock is my hand plus the board plus the opponent's hand, so the only real unknown is the opponent's hand sum O. Our bot keeps 216 hypotheses about how the opponent forms its prices (does it read our quotes, how honest are its quotes, how much edge does it need to trade), each with a Gaussian belief about O that a Kalman filter updates from every quote and answer. To act, it scores each candidate price or answer by the expected profit now plus a quick play-out of the rest of the deal under the eight likeliest hypotheses, through a square-root-like utility that matches the scoring rule. The final quote is moved 30% towards a safe price near the opponent's last quote, so we give away less information. We never peek, and we swap only in round 5, whenever the swap is expected to gain.

Q. Why use many hypotheses instead of one opponent model?

Opponents are very different: naive bots, readers, followers, hiders. Any single model is wrong against most of them. With a weighted set, the data chooses: after a few quotes, the hypotheses that predicted the opponent well carry almost all the weight.

Q. What is the Kalman filter doing?

Under each hypothesis the opponent's quote is a straight line in O (and its trust t) plus noise. For a Gaussian belief and a linear noisy reading, the Kalman filter is the exact Bayes update: move the mean by gain × surprise and shrink the variance. One honest quote can pin O down to about ±1.

Q. How do you learn anything from a pass?

A pass means the opponent's value was within its threshold of my price, which is an interval. We take the mean and variance of the belief cut to that interval (a truncated normal) and apply the same Kalman step. A buy or a sell gives a one-sided interval.

Q. What if the opponent does something strange?

Every observation has a small chance of being "unexplained": 10% for quotes, 6% for answers. One odd move then cannot destroy a good hypothesis, and the belief update is blended by the chance that the move was explained.

Q. Why 216 hypotheses, and why prune?

Five families times honesty, trade-learning, swap expectation and threshold, plus the blend quoters, gives 216. Pruning drops those 403 times less likely than the best, which keeps every call fast (median 0.2 ms locally).

Q. How is a price chosen?

In two stages. First, the price with the best immediate expected profit (including adverse selection) among offsets from the opponent's value. Then a few candidates are scored together with a play-out of the rest of the deal under the top 8 hypotheses, through the utility. Then the 30% nudge.

Q. What is adverse selection, and where is it in the code?

The opponent buys from me exactly when its hand is high, which is when V is high, so a naive expected-profit formula is too optimistic. Line 926 subtracts $\beta \cdot c \cdot \varphi(z)$, the exact correction when the belief is normal.

Q. Why a concave utility?

The score is $\text{sign}(p) \cdot \sqrt{|p|}$ per deal pair, so big wins count less than many small ones. A utility with the same shape makes the bot prefer steady profits to big gambles. We kept RISK = 60: 120 was worse and 30 was not clearly better.

Q. What is the safe quote, and why 30%?

The safe quote is the opponent's last quote, moved for new board cards and swaps, plus 1.3 towards our value. It sits inside the opponent's ±2 band, so it reveals little and is hard to trade against. Our final quote is 70% planner and 30% safe quote. Live tests: 15% gained +0.46, 30% another +0.13, nudging against every opponent another +0.06, while quoting only the safe quote lost.

Q. Why do you never peek?

A peek costs 3 points and shows one random card out of five. The opponent's quotes and answers already tell us most of O, so the information is worth less than its price.

Q. Why swap only in round 5?

By round 5 we know which way we are positioned, so we know which card to throw, and the thrown card is public with only two quotes left for the opponent to react. Moving the swap to round 5 (rio) gained +0.86 live. Since manila we swap in round 5 whenever the expected gain is positive (+0.10 live over the old bar of 4), since pamplona the decision mixes the current and the expected final position (+0.40 live), and matias weights them 0.7 / 0.3.

Q. What does the _Reader class do? Is it still needed?

It is an exact attack on the organisers' house bots (reader and sharp), which value the stock with a fixed formula. It switches on only when their quotes match that formula to the cent and off at the first surprise, so against real teams it never switches on. It is harmless, so we kept it.

Q. What happens if your code throws an error or is slow?

Every engine method has a try/except: on an error we fall back to a card-counting value (quote), a simple edge rule (answer) or doing nothing (move). marseille (manila's parent) cut the work per call by 30–40%; locally the median call takes about 0.2–0.4 ms and the slowest a few ms.

Q. Do you keep anything between deals?

No. The engine creates a fresh Bot for every deal, and everything starts again from the prior.

Q. Why are P_EXT and P_PROBE equal to 2.0?

They are probability thresholds, and a probability can never reach 2, so paid manipulation is switched off. It cost money against real teams (one probe quote lost 100 points in a deal in a local test) and turning it off was neutral to positive.

Q. What is first?

Whether we are seat 0, which quotes first in odd rounds. We learn it at our first call: if the tape is empty when we are asked to quote, we are first.

Q. What are Q and tau?

Q is the net number of lots we bought at the opponent's quotes. Some opponents raise their value when we buy from them (they conclude our hand is high); tau is how much per lot.

Q. What are kappa and ks?

Some bots expect a player who is long to swap away a low card, which raises V. kappa is how much such a bot shifts its value, and ks is the direction: the sign of our position while we can still swap.

Q. What is the difference between lam and t?

lam is about THEIR quotes: how much of their own hand shows in them (1 fully, 0 not at all). t is about OUR quotes: how much they believe what our quotes say about our hand.

Q. Why does your quote sometimes go to 0 or 195?

Only at the position limit. At +10 lots nobody can sell to us, so a quote of 195 can only be traded in our favour (they would buy at 197), and a reader takes it as a sign of a strong hand. At −10 we quote 0 for the same reason.

Q. What does the play-out (_roll) assume?

A simple deterministic future at the hypothesis' mean values: we quote 3.05 off their value when it is more than 1 away from V (so they trade our way); they trade when their edge beats their threshold; we take their quotes when V beats them by more than the 2-point spread.

Q. Why heapq.nlargest?

To pick the 8 heaviest hypotheses without sorting the whole list; it is faster.

Q. What does rr mean in _see?

The probability that the observation was explained by the hypothesis rather than being an outlier. It blends the updated belief with the old one.

Q. What are the weak spots?

Opponents that hide their hand completely and only trade on big edges give the filter little to learn from; there we rely on the nudge and the threshold hypotheses. HAN SOLO, PaperHands and Genz7 were our hardest head-to-head matchups.

Q. What would you improve with more time?

Detect the opponent's style within a deal and switch strategy per opponent: in hindsight, the best of our versions per opponent would have scored about +0.75 more than the best single version (part of that is luck), but every detector we tried misfired. More speed would also allow more hypotheses in the planner.

Your notes

12. Glossary

Term	Meaning
V	the value of the stock: the sum of both hands and the whole board
O	the opponent's hand sum (5 cards)
maker / taker	the maker names a price; the taker buys at price + 2, sells at price − 2 or passes
lot	one unit of the stock; at most 2 per trade, at most 10 held
position	lots held: + long (gains if V is high), − short
fair value	what a bot thinks V is worth
naive	an opponent that values the stock from its own cards and the public cards only
reader	an opponent that also reads what our quotes imply about our hand
blend quoter (follower)	an opponent whose quote is partly a copy of our last quote
hider / strategic quoter	an opponent whose quotes say nothing about its hand (lam = 0)
trust t	how much a reader believes our quotes
lam	how much of its own hand an opponent's quotes reveal
tau	how much an opponent's value moves per lot we buy from it
kappa	how much an opponent's value moves because it expects us to swap
take threshold	the edge an opponent needs before it trades (2 or 7 in our model)
hypothesis	one possible opponent style, with its own belief and weight
prior / posterior	belief before / after seeing the data
likelihood	how probable the observed move was under a hypothesis
log-weight	log of a hypothesis' weight; adding log-likelihoods is Bayes' rule
Kalman filter	the exact Bayes update of a Gaussian belief from a linear noisy reading
covariance	how two uncertain numbers move together (here O and t)
truncated normal	a normal distribution cut to an interval
moment matching	replacing a mixture by one Gaussian with the same mean and variance
outlier allowance	the EPS_Q / EPS_R chance that a move is unexplained
pruning	dropping hypotheses far less likely than the best
play-out (roll-out)	a quick simulation of the rest of the deal
utility	the score-like function used to compare outcomes
adverse selection	trading with someone who knows more: they trade exactly when it is bad for you
safe quote / nudge	the opponent's last quote moved 1.3 towards our value; our quote moves 30% towards it
mirrored deal	the same cards played twice with the seats swapped
paired comparison	comparing two bots on the same deals and the same opponents, so the luck cancels

Your notes

Notes

Your notes

Your notes